STATIC RANSOMWARE DETECTION IN PORTABLE EXECUTABLE FILES USING AN OPTIMIZED LIGHTGBM FRAMEWORK

Wellawattage Tharinduni Thakshila Peiris

(IT21096334)

BSc (Hons) degree in Information Technology

Specializing in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

October 2025

STATIC RANSOMWARE DETECTION IN PORTABLE EXECUTABLE FILES USING AN OPTIMIZED LIGHTGBM FRAMEWORK

Wellawattage Tharinduni Thakshila Peiris

(IT21096334)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology Specializing in Cyber Security

Department of Information Technology

Sri Lanka Institute of Information Technology

October 2025

DECLARATION

I declare that this is my own work and this dissertation1 does not incorporate without acknowledgment any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works

Signature: 

Date: 27th October 2025

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the Supervisor:

Date:

ABSTRACT

Ransomware attacks are growing rapidly and need powerful and efficient detection tools that are capable of detecting malicious executables ahead of execution time. The present paper proposes a machine learning framework for static detection of ransomware using Portable Executable (PE) header features. A LightGBM classifier optimized using Bayesian optimization proves to be exceptional with 99% precision, 99% recall, and 0.9991 AUC score. Computational efficiency is enhanced through ONNX runtime optimization, reducing the inference time to 0.16 milliseconds per file. The method addresses major challenges like class imbalance through hybrid sampling and feature redundancy through multi-criteria selection. This study confirms static analysis of PE headers is sufficient to provide discriminative power for efficient ransomware detection, offering a scalable and efficient solution suitable for integration into real-world security systems. The proposed approach significantly advances static malware detection state-of-the-art, combining high accuracy with execution realism.

Key words: Ransomware Detection, Machine Learning, Static Analysis, PE Header Features, LightGBM

ACKNOWLEDGEMENT

The process of writing this thesis has been life-changing, and I have received help from many people to whom I am grateful. Of all the people I am grateful to my supervisor Mr. Kavinga Yapa Abeywardena from the Department of Computer Systems Engineering, SLIIT. I am grateful for his advice and encouragement for this research and my development as a scholar. I would like to express my heartfelt gratitude to the Research Project team of SLIIT for their valuable sessions which helped a lot in this process. I also wish to thank the co-supervisor and presentation panel for their comments, which helped me improve the quality of the work done. To my research team, friends, and colleagues: It was a pleasure to work with you and your enthusiasm and willingness to cooperate were inspiring. You made the difficulties seem less difficult and the accomplishments more fulfilling. I would like to extend my greatest gratitude to my parents for always supporting me and believing in me in all my endeavors academically. You have been the pillar that has supported all my accomplishments.

Finally, to everyone who contributed to this project in ways both large and small: Your contribution to this work and my development is priceless and I thank you for that.

Table of Contents

DECLARATION 3
ABSTRACT 4
ACKNOWLEDGEMENT 5
List OF FIGURES 7
LIST OF TABLES 7
LIST OF ABBREVIATIONS 7
1. INTRODUCTION 8
1.1 Background literature 8
1.2 LITERATURE REVIEW 10
1.3 Research Gaps and Opportunities 22
1.3.1 Research Gap 25
1.4 RESEARCH PROBLEM 27
1.5 RESEARCH OBJECTIVES 28
2. METHODOLOGY 30
2.1 Methodology 30
2.2 Testing and Implementation 46
2.3 Commercialization Aspect of the Product 51
3. Results & Discussion 52
3.1. Results 53
3.2. Research Findings 55
3.3. Discussion 55
References 58

List OF FIGURES

Figure 1: Timeline of ransomware evolution from 1989 to present day, highlighting key ransomware families and their innovations 11
Figure 2: Geographic distribution of ransomware attacks in 2024, highlighting the most targeted countries and regions 12
Figure 3: Machine learning workflow for ransomware detection..... 14
Figure 4: Portable Executable (PE) file structure 17
Figure 5:System Diagram of Client-Server Architecture for Static Ransomware Detection System..... 31
Figure 6: Confusion Matrix for Ransomware Detection Model..... 47
Figure 7: LightGBM Feature Importance Analysis 49
Figure 8:Benign file detected..... 49
Figure 9: Ransomware file detected..... 50
Figure 10: Test Confusion Matrix 53

LIST OF TABLES

Table 1: Gap Analysis 23
Table 2: Initial Model Performance Comparison..... 37
Table 3: Bayesian Hyperparameter Optimization of..... 39
Table 4: Database 44
Table 5: Quarantine Folder 46
Table 6: Log Excerpt of Real-time File Detections 50
Table 7:Log Excerpt of Real-time File Detections 54

LIST OF ABBREVIATIONS

AUC - Area Under the Curve
DLL - Dynamic Link Library
FN - False Negative
FP - False Positive
LightGBM - Light Gradient Boosting Machine
ML - Machine Learning
ONNX - Open Neural Network Exchange
PE - Portable Executable
RA - Risk Assessment
RVA - Relative Virtual Address
SaaS - Software as a Service
TN - True Negative

1. INTRODUCTION

1.1 Background literature

1.1.1Background

In the evolving landscape of digital threats, ransomware has emerged as a particularly pernicious challenge to global cybersecurity frameworks. This kind of malware effectively holds digital assets hostage by encrypting critical data and systems, rendering them inoperable until financial demands—typically requested in cryptocurrency to permit anonymity—are fulfilled. Recent ransomware attacks demonstrate increasingly sophisticated levels of technology, employing advanced evasion tools such as polymorphic code that changes its signature constantly and advanced obfuscation methods that effectively conceal malicious activity from run-of-the-mill detection algorithms [1], [2]. The impact of such attacks extends well beyond the immediate ransom cost; organizations suffer from serious operational disruption, costly downtime, reputational damage, and significant costs of data recovery and system restoration [3].

The ever-changing nature of ransomware techniques has uncovered the built-in vulnerabilities of conventional, signature-based antiviruses. This kind of infrastructure relies on collections of known malware signatures and is, thus, inherently reactive and helpless against zero-day attacks and polymorphic varieties that alter their code with every mutation [2]. This reactive approach leaves an unsafe window of vulnerability open to new ransomware, which can execute undetected.

To combat such evolving threats, proactively smart and efficient detection mechanisms are needed at once. This research contributes to meeting this need by utilizing machine learning (ML) for static malware analysis. Static analysis involves examining the code and organization of a file without its execution, thus providing a secure and efficient first line of defense. By tapping into the wealth of data included in the Portable Executable (PE) file format—the foundation of Windows executables and libraries—this research's goal is to identify malicious patterns typical of ransomware without it ever being able to execute and cause harm.

1.1.2 Current Detection Approaches

Ransomware has established itself as a top cyber threat due to its inherent instant financial incentive for attackers. The application of Ransomware-as-a-Service (RaaS) models has made cybercrime mainstream because it facilitated even less technically savvy players to launch extremely sophisticated, large-scale attacks. High-profile ransom attacks, such as WannaCry and Colonial Pipeline, demonstrate the severe real-world consequences, including shutdowns, record-breaking financial losses, and tremendous risks to public safety [4], [5].

The reactionary nature of traditional antivirus software creates a dangerous gap in organizational security, and demands a fundamental shift towards proactive, intelligent detection systems that have forward vision and are capable of identifying emerging threats without relying on known signatures.

Several key challenges make contemporary ransomware detection difficult:

Spread of Polymorphic and Zero-Day Ransomware: Signature-based offerings, the backbone of most commercial antimalware tools, are useless against fresh or heavily obfuscated specimens until after analysis when a signature has been created and distributed. The duration is often several critical hours or days, a period during which computers are entirely vulnerable [2], [6].

The Extortionate Financial and Business Cost of Failure: The consequences of a successful attack are severe and profound. Organizations face immediate ransom payments, lengthy downtime costs, regulatory fines, irreversible data loss, and long-term reputational damage. The average ransom demand rose to approximately $2.73 million in recent years, an irrefutable indicator of the rising economic cost [5].

Resource Intensiveness of Advanced Solutions: While dynamic analysis (sandboxing) is better at detecting fresh threats, it is time-consuming and CPU-intensive. It is not possible to pass every incoming file through a resource-consumptive sandbox on an endpoint machine, thus becoming an issue of scalability for enterprise networks [7], [8].

Sophistication of Evasion Methods: Modern ransomware employs sophisticated anti-analysis methods to detect and avoid virtualized sandboxes. A sample that detects it is being analyzed in a controlled environment can delay its malicious payload or simply won't execute at all, thereby avoiding dynamic detection completely [2].

These real-world challenges create a clear and pressing demand for a new generation of defensive solutions: ones that are proactive rather than reactive; strong enough to be used in real-time on endpoints; and resilient against common evasion tactics.

1.1.3 Static Analysis of PE Files

Static analysis is the act of examining a file's properties without executing it. It is an approach enjoyed for its performance and safety as it will not execute potentially malicious code. Executable Portable Executable (PE) format, the native executable format of Windows platforms, contain deep structural information in their headers and sections that can be leveraged as indicators of malicious intentions.

It has been established through experiments that PE header fields (e.g., SizeOfCode, NumberOfSections, DllCharacteristics) can be a valuable source for distinguishing malware from benign applications [9], [10]. These works confirmed that structural metadata within the file itself contains reliable indications of malicious intent.

Machine learning algorithms are able to examine a large number of static features to identify correlations not perceivable to rule-based systems. Gradient Tree Boosting models trained

on static features have demonstrated high accuracy, with some implementations achieving precision rates of nearly 99% [11].

This research focuses on developing an optimized machine learning framework utilizing the LightGBM algorithm to analyze static PE features for ransomware detection. The goal is to create a solution with not only a high degree of accuracy but also with the computational efficiency required for real-time deployment onto endpoint systems.

1.1.4 Context and Significance of Research

The increased sophistication and rate of ransomware attacks present a clear and imminent threat to global digital security. The attacks have evolved from huge, opportunistic attacks to highly sophisticated, targeted attacks that can destroy infrastructure, healthcare, and large businesses. While existing research has advanced significantly in terms of varying detection methods, there remains an underlying and still-open challenge in the area of proactive defense.

This research addresses this challenge by developing a high-fidelity machine learning model trained on a comprehensive set of static features extracted from PE file headers. The model is designed to distinguish ransomware from harmless software with high precision, offering an active solution to the problem of ransomware. The aim is to create a thorough static analysis engine that is highly precise while being computationally efficient to a degree where it can be realistically deployed on endpoint systems, thereby adding an essential layer of defense against ransomware.

1.2 LITERATURE REVIEW

1.2.1 Evolution of Ransomware Threats

Ransomware has also changed incredibly in the last three decades, from primitive screen-locking techniques to sophisticated encryption-based extortion tools that can devastate large organizations. The AIDS Trojan of 1989, spread through floppy disks, is commonly known to be the first reported ransomware attack, using simple symmetric encryption to encrypt files and charge a $189 decryption fee [1]. This primitive method can hardly be compared with the current highly complex ransomware environment.

The ransomware development is possible with three general generations. The first-generation ransomware (1989-2005) employed primarily screen-locking techniques without actually encrypting files but rather relied on social engineering to trick victims into paying. The second-generation ransomware (2006-2013) introduced actual encryption capabilities but primarily employed weak encryption techniques which could be easily defeated by security professionals [2]. The third generation, beginning around 2013 with CryptoLocker, marked a significant leap in sophistication by implementing strong cryptographic algorithms and establishing what security researchers now term "Ransomware 2.0" [3].

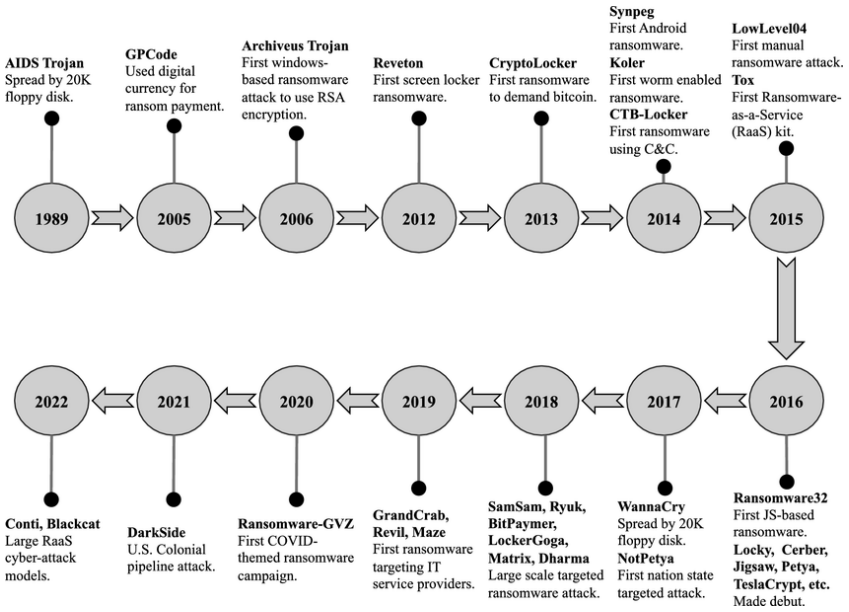


Figure 1: Timeline of ransomware evolution from 1989 to present day, highlighting key ransomware families and their innovations

CryptoLocker first used asymmetric (public-key) cryptography for encrypting files, meaning that the attackers secured their right to ransom money because victims were unable to get back their files without having the attacker's private key. It also used payment in Bitcoin, which granted attackers anonymity and allowed them to profit from their attacks [3]. These changes drastically rearranged the landscape of threats and set a model for later ransomware families to develop and elaborate on.

May 2017 WannaCry was a watershed in ransomware growth. Within a span of less than 24 hours, this historic attack compromised over 200,000 computers across 150 countries, with an estimated cost of over $4 billion [4]. Unlike its antecedents, what distinguished WannaCry was its worm-like spreading feature that utilized the EternalBlue exploit (developed by the NSA but later leaked) to replicate itself automatically across targeted networks without any user interaction [4]. This self-replication functionality transformed ransomware into stand-alone infections to network-level epidemics that could bring down entire organizations in a single blow.

Contemporary ransomware employs a variety of obfuscation techniques in multiple layers to evade detection tools. Hurtuk et al. provided an account of how newer variants employ packed executables, encrypted payloads, and sophisticated anti-analysis features that proactively defeat security tools [2]. These tactics of evasion include virtual machine detection to identify sandbox environments, sleep calls and timing checks to prevent dynamic analysis, and polymorphic code that changes its signature upon each infection [2]. Increasing sophistication of these techniques reflects an ongoing cat-and-mouse race between ransomware developers and security researchers.

Figure 2: Geographic distribution of ransomware attacks in 2024, highlighting the most targeted countries and regions

The ransomware attack surface today has expanded far beyond one computer. It targets critical infrastructure, hospitals, schools, and government agencies as the modus operandi of today's campaigns. Ransom payments averaged more than $1.5 million in 2024, an increase of 148% over the previous year, as recent data collected by Sobers indicates [5]. The healthcare sector has been particularly vulnerable, with 66% of healthcare organizations targeted by ransomware in 2024, leading to significant disruptions in patient care [5].

The financial industry has also emerged as another key target, with banking systems under considerably sophisticated attacks that exploit technical weaknesses as well as psychological vulnerabilities of humans. Tunji et al. reviewed some of the most prominent attacks on banking institutions in great detail and found that phishing email remained the primary initial infection vector, followed by exploitation of unpatched VPN vulnerabilities [3]. Their research highlighted how ransomware actors now engage in extensive reconnaissance before they drop payloads, maximizing impact to critical systems in order to maximize payment potential.

The greatest evolutionary leap in ransomware operations is the emergence of the Ransomware-as-a-Service (RaaS) business model. The model transformed ransomware from isolated assaults by individual hackers into organized criminal groups with specialized job functions and professional methods. RaaS allows technically less-skilled criminals to exploit sophisticated ransomware using pre-crafted toolkits provided by developers in exchange for a portion of the ransom, typically 20-30% [1].

Modern RaaS operations include clearly defined roles: initial access brokers who gain a foothold in the victim networks, ransomware writers who create and distribute malware, and negotiation specialists who maximize ransom payout through psychological tactics [5]. Groups such as REvil, DarkSide, and Conti have established sophisticated operational frameworks, such as technical support, affiliate programs, and escrow services. This ransomware professionalization has driven the explosive growth in frequency and economic harm, with estimated worldwide damages over $20 billion in 2024 [5].

The most frightening new trend is the creation of double and triple extortion tactics. Besides encrypting files, the attackers steal sensitive information before encryption and threaten to publish it if they are not remunerated (double extortion) [1]. There are other groups that have gone on to conduct distributed denial-of-service attacks against victims' publicly exposed

resources as a third pressure tactic (triple extortion) [1]. Such multi-faceted approaches put even more pressure on the victims to pay, despite having appropriate backups for recovery from encryption alone.

1.2.2 Conventional Detection Techniques

Conventional ransomware detection has long relied, to a great degree, on signature-based methods, which identify known malicious patterns in executable files. While reasonably effective against previously cataloged threats, these techniques necessarily break down in the face of novel or variant ransomware [6]. Signature creation typically requires manual analysis by security researchers, creating an inevitable delay between the emergence of new threats and the deployment of protection signatures [7].

Signature-based detection operates by comparing file hashes or byte patterns to databases of known malicious code. The approach enjoys a range of advantages, including minimal computational overhead and low false positive rates for known threats. However, the nature of signature generation is reactive, meaning protection only exists after malware has been discovered, analyzed, and indexed—a process that can take hours or days as infections spread rapidly [6]. Furthermore, newer ransomware employs obfuscation techniques designed to bypass signature-based detection.

Heuristic-based detection improves on signature matching by employing rule-based systems to identify suspicious attributes or behavior that are most likely to indicate malicious intent. The systems search for attributes such as unusual code structures, API calls of concern, or atypical file operations typical of malware [8]. More successful than signature-based detection at detecting zero-day threats, heuristic detection tends to have greater false positive rates due to legitimate software behavior and malicious activity resembling one another.

Medhat et al. evaluated several commercial antivirus tools with heuristic detection against a test set comprising 500 ransomware samples and 1,000 legitimate programs [6]. Their findings indicated that while heuristic detection improved detection of new variants by approximately 23% compared to signature-based techniques, it also increased false positive rates from 0.5% to 4.7% [6]. This tradeoff between detection performance and false positives is one of the biggest challenges facing security vendors who must balance protection and usability.

Behavioral analysis methods monitor the behavior of programs at runtime, generating alerts for suspicious behavior such as mass file encryption activity, destruction of shadow copies, or communication with known command-and-control servers. This approach is able to identify unknown ransomware by monitoring the actions rather than code signatures. Manavi and Hamzeh demonstrated that monitoring file system activity alone would have identified 94.3% of the ransomware samples in the test data they utilized, with most of the detections occurring within the first 10 seconds of execution [9].

While improved against novel threats, behavioral analysis techniques struggle particularly to distinguish between legitimate encryption software and ransomware, particularly security

and backup software that exhibits the same behavioral profiles as malware [9]. In addition, behavioral detection entails permitting the ransomware to execute partially before detection, which leaves open the window of potential initial damage before threat mitigation. This is particularly problematic for fast-encrypting ransomware that can cause significant harm in seconds.

Sandbox-based detection environments provide execution of questionable files in a controlled environment to observe their behavior without risking actual system compromise. These isolated environments can reveal malicious activity like attempts at encryption, registry modifications, and network communications [10]. Sandbox technology is generally employed by enterprise security products as a supplementary detection layer, particularly for analyzing email attachments and downloaded files before they reach endpoint systems.

Yet newer ransomware is employing sandbox-evasion techniques more and more that try to identify analysis environments and alter behavior to avoid them. These anti-analysis methods include searching for virtualization artifacts, monitoring mouse movement to discover automated environments, or waiting for particular user interactions to initiate [2]. Hurtuk et al. described sophisticated evasion techniques in the Cerber ransomware family, which remains dormant for an extended period when it discovers probable analysis environments, effectively bypassing time-based sandbox analysis [2].

The limitations of conventional approaches are particularly evident when facing sophisticated ransomware strains. In a controlled laboratory environment, Matin illustrated that conventional signature-based antivirus tools were capable of detecting only 68% of recent ransomware samples, with detection rates as low as 42% for samples specifically designed to evade analysis [10]. The significant detection gap highlights the pressing need for more advanced methods with the capacity to generalize to unfamiliar threats.

Further, the reactive nature of the traditional solutions provides the security vendors with an intrinsic disadvantage in the arms race with the ransomware developers. As demonstrated by Medhat et al., the average delay between the release of a ransomware variant and the availability of its signature ranged between 4 and 28 hours for major security vendors [6]. During this period, the systems remain vulnerable to infection, particularly in targeted attacks when tailored ransomware variants are employed against an organization.

1.2.3 Machine Learning-based Malware Detection

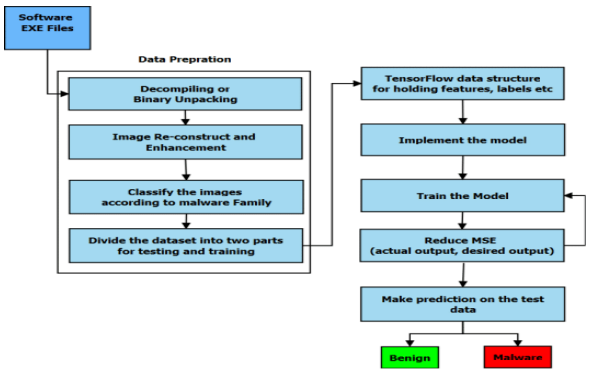


Figure 3: Machine learning workflow for ransomware detection

Machine learning has emerged as the breakthrough way of enhancing ransomware detection accuracy, far superior to conventional techniques through the ability to identify complex patterns and learn to generalize to new, unknown attacks. Compared with the signature-based method, ML systems are capable of identifying malicious software by similarities with known malware rather than exact matches, with significantly higher novel variant detection rates [7].

Supervised learning techniques have been found particularly useful in the work of ransomware classification, with several studies noting over 95% classification accuracy using well-designed features [7]. These techniques make use of labeled data samples containing ransomware and benign files for model training that is able to distinguish between the two classes based on the extracted features. Performance of these models heavily depends on both the quality of the training data as well as the discriminative power of the feature selected.

Feature extraction techniques are an integral component of ML-based malware detection systems and have a direct contribution to model performance. They can be broadly categorized into static features (derived without execution) and dynamic features (observed at runtime). Static features typically involve file header information, byte frequency histograms, imported API functions, entropy values, and structural features [8]. Dynamic features trace program activity as programs execute, including file system accesses, registry manipulations, network connections, and API call sequences [12].

Almousa et al. demonstrated that sequences of API calls from PE files can be highly discriminative features for the identification of ransomware, with 97.8% precision with random forest classifiers [7]. They investigated 2,000 Windows API calls widely employed by benign and malicious software and found a subgroup of 150 calls that provided maximum discriminative capability for ransomware detection. In the same way, Alvee et al. concluded that PE header features merged with import table analysis generated a 96.4% detection rate at a false positive rate of less than 0.8% [8].

Some of the supervised learning algorithms employed to tackle ransomware detection problems have some differing strengths and weaknesses, and the following are some of them. Support Vector Machines (SVM) have been very promising because they can determine good decision boundaries in high-dimensional feature spaces. SVM classifiers achieved a 94.2% success rate in the classification of ransomware and benign programs using static PE features, which was superior to Naive Bayes (87.6%) and Decision Tree (90.1%) classifiers using the same dataset [10].

Random Forest approaches have consistently demonstrated robust performance in various studies, with their ensemble structure providing inherent resistance to feature noise and overfitting. Research by M.J.M.M. et al. found that Random Forest was capable of 96.8% accuracy in ransomware detection with PE header features, though at greater computational requirements than some alternative approaches [11]. The inherent feature ranking of importance also provides valuable insights on the most discriminative features for detection.

Deep learning approaches have recently made a lot of progress for malware classification problems. Alvee et al. employed a deep neural network architecture with ransomware detection in industrial control systems, where they achieved a 99.1% accuracy with a minimal false positive rate compared to the traditional ML models [8]. Their multi-layer perceptron architecture consisted of three hidden layers with ReLU activation functions, which indicated improved generalization to novel samples when compared to the standard ML algorithms trained on the same features.

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were particularly promising for sequential data patterns analysis of ransomware samples. LSTM networks were demonstrated by Manavi and Hamzeh to effectively model temporal patterns of ransomware behavior and achieve 98.6% detection accuracy with static features extracted from PE headers [9]. Their approach learned sequential relationships between different features, which could not be captured by conventional ML algorithms, in order to improve detection of advanced evasive ransomware.

In spite of all these promising results, ML-based malware detection has several considerable challenges to overcome. Feature selection is a challenging issue, where high dimensionality risks causing overfitting or higher computational costs [11]. It leads to the necessity for advanced feature engineering and selection methods to identify the best subset of features, maximizing detection accuracy while keeping processing requirements low. Sustaining feature stability across varying malware families and variants remains a challenge.

Model explainability is also a significant challenge, particularly for deep learning models whose "black box" nature can complicate root cause analysis and hinder trust in security-critical applications [6]. While deep learning models are likely to be more accurate, their lack of interpretability can make it difficult for security analysts to understand why certain files were identified as malicious and may complicate incident response activities.

Furthermore, concept drift—the gradual change of statistical characteristics of target variables with time—turns into a serious issue as ransomware evolves, which can jeopardize model performance when not continually retrained [11]. M.J.M.M. et al. state that models trained with historical samples of ransomware lost on average 4-6% in performance each quarter when not continually updated, and this introduces the need for continuous learning strategies [11].

1.2.4 Static Analysis Methods

Static analysis techniques examine executable files without executing their code, which is a safe and efficient way to identify potential malware. For Portable Executable (PE) files—the typical executable format in Windows operating systems—static analysis has the objective of extracting structural information and metadata potentially revealing malicious attributes [10]. This activity has some advantages regarding ransomware discovery, including zero execution risk, lower computational overhead, and the potential to examine files prior to execution.

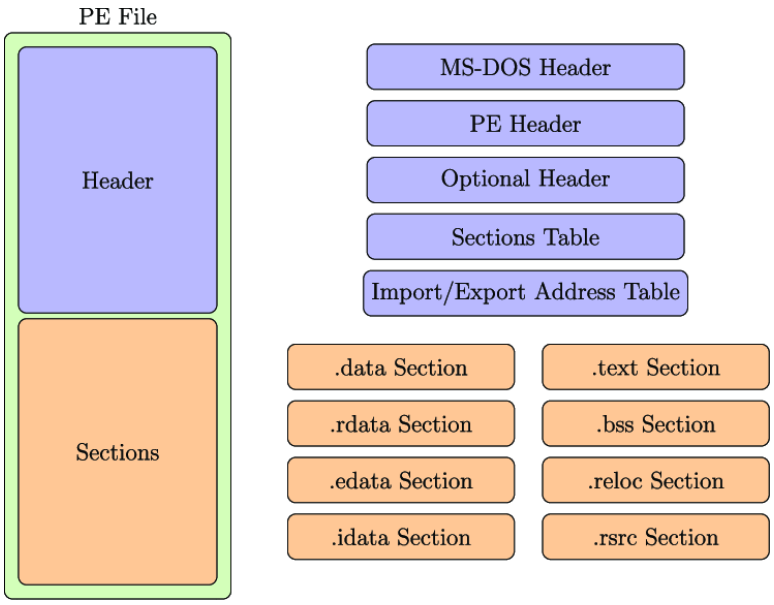


Figure 4: Portable Executable (PE) file structure

The PE file format consists of several elements including DOS header, PE header, optional header, section table, and data and code sections. All these are responsible for providing potential indicators that can distinguish between regular applications and ransomware. The static analysis techniques focus on fetching and analyzing these elements to identify suspicious patterns without requiring the files to be executed.

The PE header contains critical information about the structure of the executable, timestamp, and attributes. Matin's research showed that header attributes such as MajorLinkerVersion, NumberOfSections, and SizeOfStackReserve show statistically significant distinctions between ransomware and legitimate software [10]. On a dataset of 1,200 PE files (600 ransomware, 600 legitimate), machine learning models with only header attributes achieved classification accuracy of 91.3%, which highlights their discriminative ability [10].

These header variations reflect the various development environments and compile options typically employed during malicious software development. For example, genuine commercial applications typically utilize latest version compilers and standard compile options, while malware can utilize old version compilers to avoid certain security mitigations or unusual compile options to enable obfuscation [10].

Section properties provide an additional useful source of static attributes. PE files typically consist of sections such as .text (executable code), .data (initialized data), .rdata (read-only data), and .rsrc (resources) [9]. The number, size, and features of these sections generally differ between typical applications and malware. For instance, ransomware typically includes fewer sections than ordinary benign software with identical purpose, likely because of limited design towards malicious activities rather than the many features [9].

Entropy analysis of these sections can reveal indicators of packing or encryption, common techniques used by malware authors to obfuscate their code. Entropy measures the randomness of data, with higher values indicating greater randomness typically associated

with encryption or compression. Medhat et al. found that entropy scores of code blocks in samples of ransomware were consistently higher (mean of 7.2 on a scale of 8) than benign software (mean of 5.8), with the implication of potential obfuscation [6]. Such a strong difference is a useful heuristic for identifying potential ransomware without actually running them.

Import and export tables log the external functionality an executable relies on and the functionality it exports to other processes. These tables effectively construct a functional fingerprint of the program's capabilities and dependencies. Alvee et al. discovered in a study that ransomware samples typically import cryptographic functions (such as CryptEncrypt), file handling APIs (CreateFile, WriteFile), and system information retrieval functions that are rarely used together in legitimate software [8].

Similarly, the absence of expected imports can indicate malicious intent. Real graphical programs typically import numerous user interface functions, while their absence in an apparently graphical program could indicate malware with a deceptive icon or name [8]. Pattern-based analysis of import tables provides key information regarding the real functionality of an executable without needing to run it.

Resource section examination examines embedded information such as icons, dialog boxes, and version information. Malicious executables may contain unusual resource patterns, i.e., encrypted payloads concealed within normal-looking resources [6]. Ransomware may contain embedded text resources containing ransom notes, payment information, or even the encryption algorithms to be used during the attack [6]. Furthermore, some ransomware samples embed Bitcoin addresses within their resource sections, providing potential indications of malicious activity [10].

String and byte sequence examination can reveal hardcoded command-and-control server names, encryption keys, threatening messages, or other text artifacts of ransomware. Matin demonstrated that simple string extraction revealed suspicious text in 78% of samples analyzed for ransomware, including the strings "your files have been encrypted," "bitcoin payment," and "decrypt tool" [10]. This technique is, however, hindered by the reality that sophisticated malware now more and more employs string encryption or dynamic generation to evade detection [2].

While static analysis is a tremendous safety and efficiency gain, it does have limitations. Highly advanced packing and obfuscation techniques can make static analysis difficult, potentially hiding traces of badness. Additionally, fileless malware that operates primarily in memory but not on disk can entirely elude static analysis. But for PE-based ransomware—still the most prevalent variety—static analysis is an extremely effective and efficient means of detection.

1.2.5 Decision Trees for Classification

Gradient Boosting Decision Trees (GBDT) is a powerful class of ensemble learning algorithms that have shown to produce excellent performance in ransomware detection

tasks. Compared to a single decision tree, which can suffer from high variance and overfitting, GBDT algorithms build up a series of weak learners (shallow decision trees) in a sequential manner, where each tree attempts to correct the errors made by its predecessors [11]. This approach combines the simplicity and interpretability of decision trees with the performance advantages of ensemble methods.

The core concept of gradient boosting is to train new models to residual errors of current models. Starting with a simple initial model, the algorithm adds trees repeatedly that possess predictive strength for error residuals, enhancing the accuracy of the overall model step by step with each iteration. The process effectively reduces bias and variance in the final model and hence provides generalization performance superior to individual decision trees [13].

The three most prominent GBDT implementations—XGBoost, CatBoost, and LightGBM—are all highly visible and contain different optimizations for training speed and prediction. XGBoost, which initially brought regularization methods into the mix to control model complexity and prevent overfitting, is thus best suited for high-dimensional feature spaces typical of malware classification [11]. Its parallel processing capabilities along with cache-friendly algorithm support significantly boost the speed of training relative to standard gradient boosting.

CatBoost, developed by Yandex, introduced ordered boosting and supported categorical feature handling natively, reducing the preprocessing of PE file metadata [11]. This is particularly beneficial in the case of detecting ransomware, where several features from PE headers are inherently categorical rather than continuous. CatBoost also utilizes symmetric trees and a novel gradient calculation method that prevents overfitting with small datasets [11].

LightGBM, developed by Microsoft, utilizes two key innovations: Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). GOSS makes training more efficient by focusing on the instances with larger gradients (i.e., those which are more relevant to the learning process), and EFB reduces dimensionality by bundling mutually exclusive features [11]. These optimizations significantly reduce memory use and training time without sacrificing accuracy and hence render LightGBM particularly effective for low-resource environments.

Of all the ransomware detection, LightGBM has been most promising due to its performance. In a comparative study, Medhat et al. demonstrated that LightGBM was more effective compared to XGBoost and Random Forest algorithms for the detection of ransomware with 99.2% accuracy and 40% less inference latency [6]. This is significant in the case of real-time security operations where the time of detection directly affects the capability of the system to avert harm.

Research by M.J.M.M. et al. compared GBDT algorithms in particular for static ransomware detection and concluded that LightGBM achieved the highest accuracy (98.9%), training time, and inference speed under hardware configurations [11]. Their testing showed that LightGBM was processing feature vectors at approximately 2.8 times the speed of XGBoost

and 1.6 times the speed of CatBoost, with comparable or better accuracy [11]. These performance gains make LightGBM particularly well-suited to be deployed in endpoint security environments where computational resources might be limited.

Optimization techniques for GBDT models include hyperparameter tuning, feature selection, and ensemble techniques. Hyperparameter tuning refers to the process of adjusting algorithmic parameters such as tree depth, learning rate, and regularization strength in order to optimize performance. Bayesian optimization algorithms have been very helpful for this purpose, sampling the parameter space judiciously to maximize performance measures with minimal evaluation iteration required [11].

Feature importance analysis, which is typical for tree-based models, allows identification and selection of discriminative features with the most promising ability to differentiate attacks from benign binaries, without compromising detection accuracy [11]. For ransomware detection, M.J.M.M. et al. found that out of 50 original PE header features, there were only 15 features contributing to 95% of the model discriminativeness, which enabled dimensionality reduction with substantial savings without compromising performance [11].

Other recent advancements in GBDT optimization include quantization techniques that reduce memory requirements and inference time by representing floating-point numbers with lower precision data types [11]. This type of approach is particularly beneficial for deployment on endpoint devices with reduced computational power, enabling more ubiquitous protection against ransomware attacks. M.J.M.M. et al. experiments demonstrated 8-bit quantization reduced model size by 75% and inference time by 46% with a loss of merely 0.3% accuracy [11].

The intrinsic explainability of tree models also provides an advantage for security applications, in which understanding why a file was classified as malicious is as useful as the label itself. Decision path analysis and feature importance scores allow security analysts to know model decisions, facilitating incident response and building trust in the detection mechanism [11]. This explainability is contrary to that of deep learning techniques, which are typically "black boxes" with higher potential for accuracy.

1.2.6 Real-time Detection Systems

Effective ransomware protection, however, not only requires accurate detection models but also quick system architectures that can deliver in-the-moment security before file encryption [11]. Real-time detection systems face the double task of being very accurate and operating within strict latency demands to detect threats in time before affecting systems [11]. Designing these systems entails extremely careful selection of architectural approaches, performance optimization, system integration, database schema design, and response capacity.

Architectural implementations of real-time detection typically rely on client-server architectures that distribute computation loads among endpoints and centralized analysis

engines. As depicted by Medhat et al., a three-tiered architecture—sensor units on endpoints, feature extraction and classification analysis engines, and response systems for threat mitigation—provides a helpful manner of dividing performance and accuracy requirements [6]. The distributed design allows for minimal-monitoring processes on endpoints while leveraging more powerful server resources for intensive analysis.

In commercial settings, hierarchical designs tend to work best, with local detection in the endpoint tier complemented by more advanced analysis at gateway or cloud levels. This design allows for quick response to well-known threats while offering more detailed analysis for uncertain cases [6]. Centralized analysis also makes it easier to combine threat intelligence from multiple endpoints and potentially detect coordinated attacks that would not be evident from single device monitoring alone.

Issues of performance for endpoint deployment are preventing high resource consumption, reducing false positives to a minimum, and preventing evasion. Resource consumption is most critical since security products that consume high levels of system resources are opposed by both system administrators and end users. Optimizing techniques such as pruning, quantization, and compilation to optimized runtime forms like ONNX (Open Neural Network Exchange) can make inference latency decrease significantly without compromising on accuracy [11].

In. Christ. In production environments, these optimizations have been seen to deliver impressive performance improvements. M.J.M.M. et al. have observed that ONNX-optimized LightGBM models incur up to 30x less inference time compared to standard implementations. Previously, this allowed for millisecond-scale detection times suitable for catching ransomware before the encryption process [11]. Their experiments captured average inference latency at 0.16 milliseconds per file, enabling real-time analysis without any noticeable impact on system performance.

System integration problems include monitoring of different file operations, high-throughput support for enterprise environments, and integration with existing security infrastructures. API hooks or file system minifilters provide mechanisms for intercepting file-related operations in the kernel for analysis before potentially malicious code executes [9]. Such approaches must be carefully implemented to avoid producing performance bottlenecks or operating system instability.

In Windows systems, the most common deployment vehicle for ransomware detection is file system minifilters, which offer a good monitoring mechanism. The kernel-mode drivers within these deny I/O requests from applications to the file system to be inspected and even blocked, leading to malicious behavior [9]. Detection systems can examine executable files before their execution by hooking pre-operation callbacks of file creation and modification events, thus preventing ransomware from executing encryption routines.

Database design for malware detection systems must support both real-time query performance and longitudinal analysis capabilities. Schema designs are typically made up of tables for detection events, file metadata, and aggregate statistics to support both near-

term threat response as well as long-term trend analysis [11]. Proper indexing strategies on high-volume query columns such as file hashes and timestamps are essential to maintaining performance under heavy volumes of transactions.

An optimally designed database schema might include separate tables for file analysis results, detection events, and system metrics. The file analysis table would hold detailed information on all analyzed files like cryptographic hashes, extraction features, and classification results [11]. The events table would record individual detection events together with contextual information like device IDs and user accounts. The statistics table would hold aggregate figures like detection rates and system performance, enabling operational dashboards and trend analysis.

Alert response and handling mechanisms dictate detection results' transformation into defensive actions. Hierarchical response systems can apply various mitigation actions in function of confidence and potential impact, ranging from blocking high-confidence threats to quarantining suspicious files for further analysis [11]. SIEM system integration enables correlation of ransomware detection events with other security telemetry to provide contextual awareness for security operations teams [1].

Common response measures are file quarantine, termination of the process, and network isolation of the infected systems. Advanced response can also include automatic backup snapshot creation on detection of malicious activity, making it easier to recover in case the ransomware executes before detection is complete [1]. Some systems have "deception directories" with canary files that will trigger immediate alarms and severe containment actions when accessed, providing advanced notification of attempts at encryption [14].

In order to prevent false positives without compromising security, risk-based decision making is most commonly implemented in place of binary allow/block decisions. This approach makes probabilistic inference of confidence scores for detection results and triggers progressively more restrictive actions with rising confidence [11]. Low-confidence detections can trigger heightened monitoring, medium-confidence detections can prompt user warning and activity logging, and high-confidence detections would trigger ready-blocking and incident response processes.

1.3 Research Gaps and Opportunities

Despite significant advancements in ransomware detection techniques, several critical research gaps remain unaddressed. Current methodologies face limitations in various dimensions, creating opportunities for innovative approaches that can enhance detection capabilities while addressing practical deployment considerations. These gaps span theoretical foundations, methodological approaches, and implementation challenges.

Table 1: Gap Analysis

Gaps in Current Methodologies	Industry Standard Approach	Proposed Solution	Gap Addressed
Limited integration of static and dynamic analysis	Separate qualitative and quantitative analyses operating independently	An integrated framework combining static PE features with optimized ML classification	Integration of complementary analysis methods for higher accuracy
High computational overhead in real-time detection	Resource-intensive analysis requiring dedicated hardware or cloud processing	Lightweight, ONNX-optimized models designed for endpoint deployment	Practical deployment on standard endpoint devices
Insufficient feature engineering	Generic malware features without ransomware-specific optimization	Multi-criteria feature selection focused on ransomware discriminative power	Higher detection accuracy for ransomware-specific threats
Inadequate handling of class imbalance	Standard sampling techniques or basic weighting approaches	Hybrid sampling with focused augmentation of minority class instances	Improved model performance on imbalanced real-world data
Poor model explainability	Black-box models providing limited insight into detection rationale	Transparent classification with feature importance visualization	Enhanced trust and explainability for security operations
Limited performance in real-world deployment	Laboratory-optimized models that degrade in production environments	Production-optimized framework with inference-time performance focus	Maintained detection efficacy in real-world environments

One key flaw in current research is the artificial separation between static and dynamic analysis techniques. While each method has specific advantages, their segregation leads to inevitable blind spots in detection coverage. Static analysis is effective and pre-execution detection efficient but is susceptible to sophisticated packing techniques. Dynamic analysis is effective in detection of actual malicious intent but has partial execution and higher computation overhead [12].

While Netto et al. proposed an integrated methodology combining both techniques [15], the process had sequential processing with considerable latency, making it unsuitable for real-time defense [15]. One possible is to develop truly parallel analysis paradigms that leverage the strengths of each methodology without compounding its computational cost. That would provide complementary detection, where static analysis is a light first-pass filter and dynamic analysis provides more penetration for ambiguous instances.

Feature engineering is another area that has a lot of potential for growth. Most existing work is based on generic malware features rather than features specifically engineered and optimized for ransomware detection [10]. This fails to utilize the inherent behavioral and structural features that are specific to ransomware and distinguish it from other forms of malware. For instance, ransomware possesses distinct file access patterns, encryption patterns, and resource usage profiles which can be highly discriminative features if well designed.

Gulmez et al. highlighted this limitation in their comparative analysis, showing that even deep learning models of high levels performed badly when they were trained on bad feature sets [12]. Experiments showed by the authors that models trained using only general malware features achieved only 83% accuracy when particularly applied to ransomware, while models trained using ransomware-specific feature sets achieved 97% accuracy [12]. Such a huge variation in performance reflects how important specially crafted feature engineering is to effective ransomware detection.

Class imbalance is another principal issue, primarily for supervised learning techniques. Benign files disproportionately overshadow ransomware samples in real-world scenarios, generating inherent imbalances during training of models [13]. Standard approaches like random oversampling or undersampling do not tend to effectively resolve this imbalance and therefore result in models biased towards the majority class or overfitting on artificially created minority samples.

More sophisticated techniques like hybrid sampling techniques, class-weighted loss functions, or ensemble techniques specific to imbalanced data might be able to provide better detection performance in real-world conditions [13]. Experiments by M.J.M.M. et al. demonstrated that hybrid sampling techniques involving SMOTE (Synthetic Minority Over-sampling Technique) and removal of Tomek links improved F1 scores by 6-8% compared to regular sampling techniques in dealing with extremely imbalanced datasets typical of operational conditions [11].

The accuracy versus performance trade-off problem is probably the most critical remaining research gap. Computational resources higher than levels realistically available to endpoint systems in laboratory experiments are commonly used, and this renders reported results unrepresentative of actual effectiveness [11]. Such a gap commonly leads to disappointing performance after deploying theoretically exciting approaches in production systems.

Studies by Medhat et al. revealed that 99% accurate models on benchmark testing when deployed in the field on typical endpoint hardware incurred an average of 15-20% loss of performance [6]. The loss of performance resulted from a variety of factors including resource constraints, diverse file types, and the occurrence of valid but unforeseen applications that led to false positives. The gap between field and lab performance highlights the need for more realistic testing approaches that better reflect actual deployment conditions.

There is potential for developing deployment-focused strategies that maximize inference-time performance and training-time accuracy. Such strategies would include optimization techniques geared towards the resource-constrained environments of endpoints, including model quantization, pruning, and compilation to hardware-accelerated forms [11]. These optimizations are a rich area for future work, with potential for high-performance ransomware detection without the requirement of specialized hardware.

Static analysis approaches have particular promise to be optimized as they avoid the overhead of observing behavior or sandboxed execution. With high-discriminative features extracted from PE headers and efficient classification methods, static analysis approaches might be able to achieve the best trade-off between speed and accuracy to guarantee efficient ransomware prevention [10]. This approach is part of the overall industry trend of shifting security controls "left" in the attack sequence, from detection and response to prevention.
Explainability is another predominant research topic, particularly for security application scenarios where understanding why something was detected is at least as useful as the detection. While deep learning approaches were very accurate in some research, their "black box" nature makes it difficult to incident respond and may reduce system trust [6]. Research on explainable AI techniques specifically for security application scenarios may bridge this gap, providing high accuracy with interpretability.

Integration with threat intelligence models also presents a possible path to innovation. Existing solutions for detecting ransomware are typically cut off from tapping the significant volumes of external threat intelligence to be found in security providers, information sharing groups, and open-source intel [1]. Research into how best to integrate this external intelligence into detection models could potentially more effectively hone accuracy and responsiveness to emerging threats.

In total, a great deal has been done towards ransomware detection with machine learning techniques, but there is enormous space for work to address the deployment, optimization, and operational realism issues. Resolving these shortcomings will allow the follow-up work to bridge the gap from theoretical possibilities to practical reality of defending against the ever-changing ransomware threat environment.

1.3.1 Research Gap

Despite significant advances in malware detection research, several critical gaps remain in the specific domain of ransomware detection in PE files:

- **Static Model Adaptability Limitations**

Existing static analysis frameworks demonstrate limited adaptability when confronting new ransomware variants. While many models perform adequately on known samples, they struggle significantly with the detection of novel variants or those from emerging ransomware families. Manavi and Hamzeh found that traditional static models experienced a 22-37% reduction in detection accuracy when tested against ransomware variants not included in their training data [9]. This adaptability gap is particularly problematic given the accelerating evolution of ransomware techniques and the growing sophistication of attack methods.

A critical but overlooked dimension is the accuracy-robustness trade-off in static ML models. Models that achieve impressive metrics in controlled laboratory environments often collapse when confronted with real-world obfuscation techniques and class imbalance. Gulmez et al. demonstrated that ML models achieving 98% detection accuracy in balanced

test environments saw performance drop below 76% when confronted with heavily obfuscated samples and realistic benign-to-malicious ratios [12]. This fundamental gap between theoretical and operational performance undermines confidence in research solutions.

- **Insufficient Understanding of Feature Evolution**

There exists limited comprehensive research on how static feature patterns evolve across ransomware families and generations. While Matin et al. demonstrated that PE header features can provide reliable indicators of malicious intent [17], the evolutionary patterns of these features across ransomware variants remain insufficiently understood. This gap in understanding prevents detection systems from anticipating how ransomware PE structures might evolve, creating blind spots in detection capabilities.

The methodological limitations in testing approaches further compounds this issue. Current research lacks frameworks for continuous feature engineering that adapt to emerging threats. As noted by Netto et al., "traditional testing methods fail to account for the rapid evolution of features across ransomware generations, leading to models that quickly become obsolete against novel threats" [15]. The absence of systematic methods for perpetually updating feature selection and model training represents a fundamental limitation in current approaches.

- **Evasion Technique Analysis Deficiencies**

Current research inadequately addresses the progression of evasion techniques in ransomware development. As noted by Almousa et al., "the rapid evolution of anti-analysis techniques in ransomware consistently outpaces the development of corresponding detection methods" [7]. This widening sophistication gap ensures that detection mechanisms do not take into account the latest obfuscation tactics, and ransomware can bypass detection despite methods known to the security community.

- **Missing Early Detection Capabilities**

Most critically, there is a significant gap in methods for detecting emerging ransomware threats during the crucial first moments after a file appears on a system. Alvee et al. emphasized that "the window for effective ransomware intervention is extremely narrow, requiring detection within seconds of introduction to prevent encryption processes from initiating" [8]. Most solutions today favor post-execution identification over pre-execution prevention, leaving the systems exposed during the initial few seconds when mitigation would be most effective.

The computational intractability of dynamic analysis exacerbates this challenge. While dynamic analysis provides superior detection capabilities, its resource requirements make it impractical as a scalable first-line defense. Research by Medhat et al. demonstrates that comprehensive dynamic analysis typically requires 15-30 seconds per file [14], far exceeding the 3-7 second window before encryption begins. This fundamental timing constraint necessitates lightweight static analysis approaches capable of near-instantaneous detection.

By occupying these specific research niches, this research aims to develop an optimized LightGBM model for static ransomware detection that can identify threats within the first 1-5 seconds of a file's existence on a system, before execution is possible, with high accuracy on both new and current ransomware variants.

1.4 RESEARCH PROBLEM

1.4.1 Problem Statement

The uncontrolled exponential growth of ransomware has confronted cybersecurity infrastructures with an unprecedented threat, particularly in dealing with obfuscated versions within Portable Executable (PE) files. Ransomware has matured from basic encryption utilities to sophisticated threats that are capable of bypassing traditional detection pathways [2]. The damage caused by these attacks continues to escalate, with global financial impacts exceeding $20 billion annually according to recent cybersecurity reports [5].

The specific problem addressed in this research is the inadequacy of current static detection methods to identify modern ransomware embedded in PE files before execution. This critical limitation exists for several key reasons:

First, traditional signature-based detection approaches fail to identify novel or modified ransomware variants. These methods rely heavily on known patterns that can be easily altered by attackers, creating a continuous cat-and-mouse scenario where defenders remain one step behind [6]. A study by Nayak et al. demonstrated that signature-based methods detected only 62% of modified ransomware samples in controlled tests [1].

Second, current static analysis techniques lack the sophistication to detect ransomware employing advanced obfuscation. Hurtuk et al. demonstrated how modern ransomware utilizes multiple layers of code obfuscation that cause malicious code to appear legitimate to many analysis tools [2]. Their research showed that obfuscation techniques reduced detection rates by up to 43% across multiple commercial security products.

Third, many existing cybersecurity solutions rely excessively on dynamic analysis (behavioral monitoring), which only identifies threats after execution has begun. As noted by Medhat et al., "by the time behavioral indicators are detected, critical system files may already be compromised" [6]. This reactive approach creates an inherent vulnerability window where ransomware can initiate encryption processes before detection occurs.

Fourth, the expanded attack surface in modern computing environments exacerbates these challenges. With the proliferation of endpoints, cloud services, and file-sharing mechanisms, ransomware has an unprecedented number of attack vectors [16]. This vast surface area necessitates highly scalable, automated protection mechanisms capable of analyzing

thousands of PE files daily across diverse environments yet maintaining near-zero performance impact – a requirement unmet by current solutions.

Finally, the detection of ransomware must occur within an extremely narrow timeframe—ideally within the first few seconds after a PE file appears on a system. Research by Tunji et al. established that modern ransomware begins file encryption processes within 3-7 seconds of execution [3], highlighting the critical need for near-instantaneous detection capabilities.

1.5 RESEARCH OBJECTIVES

The increasing prevalence and depth of ransomware assaults render it imperative to develop effective, robust detection tools that are capable of identifying malicious executables before runtime execution. Traditional signature-based detection techniques become severely handicapped in the presence of zero-day mutations, polymorphic code, and complex obfuscation techniques deployed by modern ransomware families. Here, the objectives of this research are enumerated clearly. Through these objectives, the research develops a well-defined methodology and provides metrics by which the success of the proposed ransomware detection model can be measured.

1.5.1 Primary Objective

The primary objective of research is to design a good static analysis model that can detect ransomware in PE/DLL files. Detection using current methods is usually not very effective when the malware writers apply obfuscation techniques to conceal their code. This research aims to devise a method that is effective even in such concealed attacks.

The main aim of this research is:
To build a static analysis model that is effective in identifying ransomware in PE/DLL files even if the files have been obfuscated and to overcome the inefficiencies that make traditional detection ineffective.

1.5.2 Secondary Objectives

In an effort to achieve the main objective, four key supporting objectives have been established:

- **Accuracy Enhancement**

The research seeks to improve detection accuracy through a focus on specific PE header features that can help distinguish between clean programs and ransomware. PE headers contain significant information about a file's structure and behavior that can encode malicious intent. Through careful header inspection, the research can identify patterns characteristic of ransomware but rare in regular applications.
This approach is that of looking for the most reliable indicators out of these headers and developing methods for extracting and analyzing them in an efficient manner. The

objective is to develop a model for classification that is capable of classifying ransomware accurately.

- **Lightweight Design**

A security solution is not much good unless it's possible to deploy. Most existing detection schemes are computationally expensive and make extensive use of system resources, and so are difficult to deploy on real systems. This work aims to create a lightweight model which:

- Processes files quickly without relying on excessive hardware
- Makes minimal use of system resources while running
- Can be installed on various platforms without affecting performance
- Is light enough for practical deployment in real-world security systems

- **False Negative Reduction**

Prevention of false negatives – when ransomware is not being detected – is one of the biggest challenges in malware detection. This is especially problematic for obfuscated variants designed to evade detection. The work will:

Build techniques to significantly counter these missed detections
Address specifically the detection of popular approaches to obfuscation used by authors of ransomware
Build techniques that can learn and adapt to novel obfuscation mechanisms as they emerge
Test extensively against well-known obfuscated samples to validate effectiveness

- **Efficient Implementation**

Finally, the research focuses on the operational implementation aspects of the detection system:

- Constructing a streamlined detection procedure that eliminates extraneous steps
- Constructing a flexible structure that can be adapted as new threats emerge
- Designing efficient data handling methods for processing large volumes of files
- Establishing clear metrics to measure how well the system performs in actual situations

These objectives are the foundation of the study. By addressing each of them methodically, this study hopes to develop a ransomware detection utility that offers significant improvement over existing methodologies while being practical enough for practical application.

2. METHODOLOGY

2.1 Methodology

This research employs an end-to-end machine learning-based approach to create a production-grade static ransomware detector for endpoint protection applications. It incorporates meticulous data science pipelines, advanced feature engineering practices, comparative algorithmic evaluation, and systems-level implementation considerations to achieve a solution that optimizes exceptional detection capabilities with deploy ability. Its methodology is predicated on the idea that effective ransomware detection must meet a number of conflicting requirements at the same time, including detection precision, computational overhead, false positives minimization, and integration with already deployed security hardware and software. Unlike purely theoretical techniques that are tailored for single metrics, this method focuses on developing a generalizable solution that can work stably in practical production environments where performance metrics such as inference latency, memory consumption, and system resource provisioning are all equally critical to detection accuracy.

The methodology advances through an organized sequence of mutually dependent phases with each phase emerging from the constituent parts of the previous one. Commencing with careful dataset preparation and quality control, the research advances through sophisticated feature engineering, contrasting model identification, hyperparameter optimization, performance enhancement through sophisticated conversion processes, and ultimately culminates in complete system design implementation from client-server communication protocols to database persistence mechanisms and security-focused operational controls. This integrated approach ensures that each component of the final system has undergone stringent testing and tuning for its own function in the bigger detection pipeline. Research methodology adheres to sound cybersecurity research ethics standards across all experimental procedures, specifically emphasizing safe handling of evil samples, containment of potential threats during analysis, and replicability of experimental results through controlled laboratory facilities. Furthermore, the methodology involves validation against practical operating limitations such that theoretical performance gains have a useful translation to realistic deployment conditions.

2.1.1 Research Design and Experimental Framework

The fundamental research design implements a multi-stage experimental configuration characterized by iterative refinement cycles and comprehensive validation protocols at each developmental milestone. The proposed system represents an end-to-end detection pipeline specifically architected for endpoint monitoring configurations, where individual workstations and user devices must be protected against ransomware infiltration through multiple infection vectors including downloads from internet sources, file transfers via email attachments, executable files copied from external storage devices such as USB drives, compressed archives that extract potentially malicious executables, and peer-to-peer file sharing mechanisms. The system architecture deliberately emphasizes static analysis methodologies over dynamic execution-based approaches due to several critical advantages including elimination of execution-related risks, significantly reduced analysis

latency enabling real-time detection within milliseconds rather than the seconds or minutes required for behavioral analysis, and enhanced scalability allowing processing of thousands of files simultaneously without the resource overhead associated with sandboxed execution environments.

Figure 5 illustrates the overall system architecture behind this work, the distributed client-server model deployed to support real-time ransomware detection on endpoint devices. The architecture splits the responsibilities into light-weight client-side monitoring agents and a server-side analysis engine. On the client side, the file detection module monitors system directories constantly for files of type PE/EXE/DLL that are being updated or created. Once a qualifying file is detected, the system then performs a preliminary check to determine whether it is compliant with PE file format specifications or not; non-PE files bypass the analysis pipeline to conserve resources. For PE files, the feature extraction module inspects the portable executable structure to derive the 16 static features required for classification. These features that have been extracted are reported over network communication to the server side, where there is a trained machine learning model. The ML model prediction module within the server generates a classification output regarding whether or not the file exhibits ransomware-like behavior. Where the prediction indicates potential ransomware, the system performs a secondary whitelist scan to prevent false positives on known-safe system files such as Windows system executables or code-signed applications from trusted publishers. Files that clear the whitelist scan trigger immediate security notifications to notify administrators of recognized threats, and ransomware detected proceeds into quarantine procedures to prevent execution. Conversely, files categorized as harmless proceed without disruption. Regardless of classification outcomes, all analysis procedures are completely logged to the centralized database, creating an audit trail for forensic analysis, security operations monitoring, and long-term threat intelligence accumulation. This design supports transparent security protection that runs continuously without any intervention from users while also keeping precise records of all the detection operations.

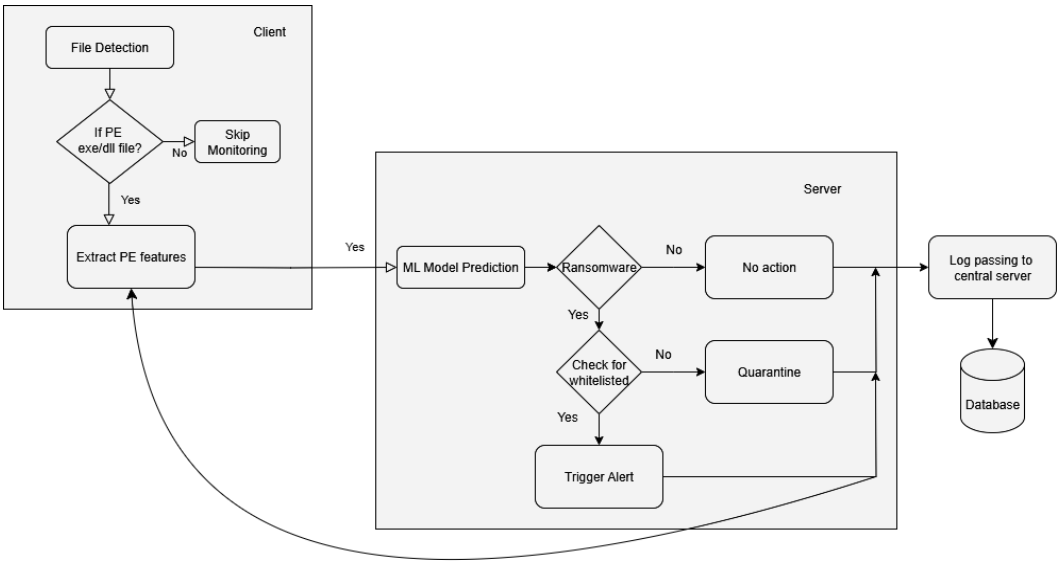


Figure 5:System Diagram of Client-Server Architecture for Static Ransomware Detection System

The detection pipeline implements a layered architecture consisting of distinct functional components that operate in coordinated sequence. File acquisition mechanisms continuously monitor specified system locations for the appearance of Portable Executable format files including standard executable files with .exe extensions, Dynamic Link Library files with .dll extensions, and any other PE-format binaries regardless of file extension. Upon detection of a qualifying file, the system immediately initiates the feature extraction subsystem which performs comprehensive parsing of the PE file structure to extract discriminative static characteristics from file headers, section tables, import address tables, export directory tables, resource sections, and optional header fields. These extracted features are subsequently transmitted to the machine learning classification subsystem which applies pre-trained predictive models to generate maliciousness probability scores and classification decisions. Finally, detection results along with comprehensive metadata are forwarded to centralized logging infrastructure to enable security operation center visibility, forensic investigation capabilities, and long-term threat intelligence aggregation.

The research design incorporates theoretical foundations from multiple disciplines within computer science and cybersecurity. From a machine learning perspective, the methodology applies supervised learning paradigms where models learn discriminative patterns from labeled training examples consisting of verified benign software and confirmed ransomware specimens. The feature extraction approach draws upon principles of digital forensics and malware analysis, specifically leveraging the reality that executable file structure characteristics provide reliable indicators of malicious intent even in the absence of runtime behavioral observation. The system architecture design incorporates established software engineering patterns including client-server separation of concerns, modular component design enabling independent optimization and replacement of subsystems, and asynchronous communication protocols that prevent blocking operations from degrading system responsiveness. Security-by-design principles permeate the architecture through multiple defensive layers including file source classification, whitelist-based filtering to prevent analysis of known-safe system files, and graduated response mechanisms that escalate actions based on threat confidence levels.

2.1.2 Dataset Acquisition and Curation Procedures

The research initiated with comprehensive dataset acquisition activities focused on obtaining a representative sample of both benign and malicious executable files suitable for training robust classification models. The initial dataset comprised 62,484 distinct PE/DLL file samples characterized by 18 static features extracted from Portable Executable file format structures. These features were systematically derived from multiple components of the PE file specification including the DOS header which provides backward compatibility information, the PE header containing fundamental file characteristics such as target architecture and timestamp information, the Optional Header supplying critical fields including subsystem type and image characteristics, the Section Table describing memory layout and individual section attributes, the Import Directory Table enumerating external dependencies, the Export Directory Table listing functions

provided by the executable, and the Resource Section containing embedded assets such as icons and version information. The dataset composition deliberately balanced representation of different file types including end-user applications, system utilities, library files, installer packages, and malicious specimens spanning multiple ransomware families to ensure model training exposure to diverse manifestation patterns of both benign software and ransomware variants.

The initial dataset underwent rigorous quality control procedures to identify and rectify data integrity issues that could compromise model training effectiveness. Comprehensive validation protocols scanned for missing feature values that would prevent proper model training, identified inconsistent or impossible feature combinations indicating corrupted PE structure parsing, and detected duplicate samples that would artificially inflate training set size while providing no additional discriminative information. The duplicate removal process implemented cryptographic hash comparison to identify binary-identical files regardless of filename or path differences, eliminating redundant samples that could skew model learning toward overrepresented specimens. Additional validation procedures verified PE header structure integrity to ensure extracted features accurately represented actual file characteristics rather than artifacts of parsing errors. Files exhibiting structural anomalies preventing reliable feature extraction were excluded from the dataset to maintain consistent feature representation across all samples.

Upon completion of quality control procedures, the refined dataset consisted of 32,255 high-quality samples characterized by 16 optimally selected features, representing a reduction from the initial 18 features through elimination of low-information content attributes that contributed negligible discriminative power. This dataset reduction maintained essential representative diversity while improving data quality through removal of problematic samples. The final dataset composition exhibited realistic class imbalance patterns characteristic of operational cybersecurity environments, containing 21,573 benign samples (representing 66.9% of total data) and 10,682 malicious samples (representing 33.1% of total data). This approximately 2:1 ratio of benign to malicious samples reflects realistic operational distributions where legitimate software substantially outnumbers malware specimens, necessitating model training approaches robust to imbalanced class distributions. The deliberate retention of class imbalance in the training data, rather than artificially forcing balanced class sizes, ensures that trained models develop appropriate decision boundaries reflective of real-world probability distributions, reducing false positive rates in deployment scenarios where the overwhelming majority of encountered executables represent legitimate software.

Dataset provenance and collection methodology adhered to rigorous ethical standards for cybersecurity research. Malicious samples were obtained from established malware repositories maintained by security research organizations, ensuring access to verified ransomware specimens while maintaining appropriate containment protocols. All experimental procedures involving malicious files were conducted within isolated laboratory environments implementing network segregation to prevent inadvertent malware propagation, system virtualization to enable safe manipulation of potentially dangerous files, and comprehensive logging to maintain audit trails of all research

activities. Benign sample collection drew from diverse sources including open-source software repositories, commercial application vendors, and system utility collections to capture representative diversity of legitimate software ecosystems. Particular attention was devoted to including samples from various software categories including productivity applications, development tools, system utilities, multimedia software, and business applications to ensure model exposure to comprehensive benign software characteristics.

2.1.3 Feature Engineering and Selection Methodologies

Feature engineering constitutes a critical phase in the development of effective machine learning-based malware detection systems, as the quality and discriminative power of input features directly determine the theoretical ceiling of achievable detection accuracy. This research implemented comprehensive feature extraction procedures specifically designed to capture discriminative characteristics of PE file structures that differentiate ransomware executables from legitimate software. The feature extraction approach focused on static attributes derivable from PE file format structures without requiring code execution, including numerical fields from various header structures, calculated entropy values indicating compressed or encrypted content, section count and size statistics, and characteristics of import/export tables reflecting external dependencies and provided functionality. This static analysis approach offers multiple advantages over dynamic behavioral analysis including elimination of execution risks associated with running potentially malicious code, dramatically reduced analysis latency enabling real-time detection suitable for endpoint protection scenarios, and enhanced scalability permitting parallel analysis of multiple files simultaneously without sandbox resource constraints.
The specific feature set engineered for this research reflects careful selection of PE file attributes proven to exhibit discriminative power for distinguishing malicious executables from benign software. The Machine field extracted from the PE header specifies the target processor architecture and can reveal anomalies such as mismatches between declared architecture and actual code structure. DebugSize and DebugRVA fields from the optional header provide information about embedded debugging data, with unusual values potentially indicating packed or obfuscated malware. MajorImageVersion and MajorOSVersion fields specify version information that may exhibit suspicious patterns in malware specimens. ExportRVA and ExportSize characterize the export directory table which enumerates functions provided by the executable, with unusual export patterns potentially indicating malicious functionality. IatVRA captures characteristics of the Import Address Table which governs runtime linking to external libraries, with import patterns differing substantially between legitimate software and ransomware due to the specialized API functions required for file encryption and system manipulation. MajorLinkerVersion and MinorLinkerVersion identify the toolchain used to build the executable, potentially revealing malware compiled with unusual or outdated development tools. NumberOfSections indicates structural complexity through section count, with ransomware sometimes exhibiting abnormal section configurations. SizeOfStackReserve specifies memory allocation parameters that may differ between benign software and resource-intensive malware operations. DllCharacteristics captures security-related flags such as Address Space Layout Randomization and Data Execution Prevention support, with malware potentially exhibiting anomalous security feature combinations.

ResourceSize measures the size of embedded resources, with unusually large resource sections sometimes indicating payload embedding. BitcoinAddresses represents a specialized feature detecting embedded cryptocurrency addresses commonly present in ransomware for ransom payment collection.

Following initial feature extraction, the research implemented rigorous feature selection procedures to identify the subset of features providing optimal discriminative power while eliminating redundant or low-information attributes that contribute computational overhead without improving classification accuracy. Multiple complementary feature selection methodologies were employed to ensure robust identification of optimal feature subsets through convergence of independent selection approaches. Chi-Square statistical testing evaluated the statistical independence between individual features and target classifications, with high chi-square scores indicating strong dependency relationships suitable for classification purposes. This statistical approach quantifies the extent to which feature value distributions differ between benign and malicious classes, with larger differences indicating greater discriminative utility. Recursive Feature Elimination implemented an iterative backward selection approach that trains models on progressively reduced feature sets, eliminating features contributing least to classification performance at each iteration until reaching an optimal feature subset. This approach accounts for feature interactions and redundancies by evaluating features within the context of other selected attributes rather than in isolation.

Lasso regression-based feature selection applied L1 regularization penalties during model training, automatically driving coefficients of uninformative features toward zero and effectively performing embedded feature selection as an integral component of the training process. This approach offers the advantage of simultaneous model training and feature selection, with regularization strength controlling the aggressiveness of feature elimination. Random Forest feature importance scoring leveraged ensemble tree-based models to quantify feature discriminative power through measurement of each feature's contribution to prediction accuracy and node purity across the ensemble of decision trees. This approach captures non-linear feature contributions and interaction effects that linear methods might miss. The convergence of multiple independent feature selection methodologies toward similar feature subsets provided robust validation of selected features, as attributes consistently identified by diverse selection approaches demonstrate reliable discriminative power independent of any particular selection methodology's assumptions or limitations.

Beyond feature selection, the research implemented comprehensive data preprocessing procedures to ensure optimal model training conditions. Multiple scaling strategies were systematically evaluated to identify optimal feature normalization approaches for the specific characteristics of PE header features. Standard Scaler implements z-score normalization that transforms features to zero mean and unit variance, providing a standardized scale across features with different native ranges while preserving outlier information. Min-Max Scaler applies linear scaling to map feature values into a bounded range typically zero to one, ensuring no feature dominates purely due to scale differences while potentially compressing outlier distributions. Robust Scaler employs median and

interquartile range-based scaling that provides resistance to outlier influence, particularly valuable when feature distributions contain legitimate outlying values that should not be suppressed. Systematic evaluation across all scaling approaches through cross-validation procedures identified optimal preprocessing strategies for the specific distributional characteristics of PE header features, with final scaler selection based on comprehensive performance measurement across multiple classification algorithms and evaluation metrics.

2.1.4 Addressing Dataset Imbalance with Sampling Strategies

The class imbalance present in the curated data set, representative of real-world operational conditions where benign software overwhelms malware specimens by many folds, needed sophisticated handling strategies to prevent model bias against the majority class at the expense of correct minority class detection. Naive training from class-imbalanced data usually results in models that achieve high general accuracy through the prediction of the majority class for most examples but overlook valid malware samples due to inferior learning of minority class characteristics. This research employed advanced techniques specifically designed to address class imbalance without losing model generalization capability and preventing overfitting to artificially balanced training sets.

Synthetic oversampling techniques were employed to generate additional minority class instances that augment the quantity of ransomware patterns in the training set. Rather than simple copycat duplication of existing malicious samples that would not provide anything new and may cause overfitting to some specimens, synthetic oversampling techniques generate new cases by interpolation between existing minority class instances. Specifically, the Synthetic Minority Over-sampling Technique generates new cases by selecting random sets of the minority class and creating synthetic instances along line segments between neighbors in feature space, thus expanding the decision region of the minority class without having unnatural feature value combinations. Appropriate deployment of synthetic oversampling enhances minority class representation to achieve well-balanced training sets in which the model receives sufficient exposure to ransomware structures to learn good discriminative features.

Complementary undersampling methods utilized intentional reduction of majority class sample sizes to prevent overwhelming minority class patterns when training the model. Rather than indiscriminate majority class removal that could happen to eliminate informative benign examples, well-informed undersampling techniques preserve majority class instances near decision boundaries or define diverse regions of the benign class distribution with care. Undersampling techniques strike a balance between prudence and the possibility of eliminating informative examples that capture the entire diversity of benign software attributes. By combining undersampling to reduce majority class oversaturation with oversampling to enhance minority class presence, more well-balanced training sets are generated with the characteristic patterns of both classes preserved.

Along with sampling-based approaches, the research incorporated class weighting techniques that adjust the loss function at model training time to enhance the penalty for minority class misclassifications relative to majority class misclassifications. With class frequency weights inversely proportional, the training process naturally offsets the influence of each class in model parameter adjustment regardless of disproportionate

sample numbers. The technique offers the advantage of not changing actual dataset structure while emulating similar effects as physical resampling through computational model training compensation. The hybrid approach involving strategic sampling and class weighting performed better by leveraging the complementary advantages of the two methods, with detailed inspection through cross-validation methods to ensure that imbalance handling techniques improved overall detection performance rather than introducing artifacts or compromising generalization to out-of-sample data.

2.1.5 Machine Learning Model Development and Comparative Evaluation

The development phase used intense testing of different machine learning approaches to identify better techniques for PE-based ransomware detection by using intensive comparative evaluation. Rather than proceeding towards an individual algorithm, the research followed extensive empirical testing across different learning paradigms to realize the identification of genuinely better methods based on empirical performance rather than hypotheses. The compared algorithms span numerous machine learning families including tree-based, ensemble, and gradient boosting schemes with strengths varying on interpretability, training speed, prediction speed, and detection accuracy.

Decision Tree models were employed as simple baseline classifiers that yielded interpretable decision rules in terms of hierarchical feature-based partitioning of the sample space. The recursive partitioning approach constructs the tree structure wherein every internal node performs a test on a specific feature value with test results appearing as branches and leaf nodes holding classification decisions. Decision Trees offer the valuable advantage of interpretable decision logic permitting direct human interpretation of classification rationale, thereby being suitable for determining which PE features most significantly indicate malicious intention. But individual decision trees are prone to high variance and overfitting bias when trained to deep depth to capture complex patterns. Baseline accuracy of 98.57% on test set was achieved by the first Decision Tree implementation, which indicates even the simplest tree-based models learn a huge amount of discriminative information from PE features, and also indicates potential for further improvement with more sophisticated ensemble and boosting methods.

Table 2: Initial Model Performance Comparison

Model	Accuracy	Precision	Recall	Inference Time
Decision Tree	96.99%	99.13%	96.34%	2.1ms
Random Forest	97.94%	99.45%	97.46%	15.2ms
XGBoost	98.90%	99.46%	98.88%	8.7ms
LightGBM	98.87%	99.42%	98.88%	5.02ms

Random Forest ensemble methods significantly enhanced prediction robustness by aggregating the predictions of numerous decision trees learned on bootstrap samples of the data. Random Forest constructs a multitude of individual trees on random subsets of training samples and random subsets of attributes at each decision of splitting, then takes majority vote predictions to produce final classifications. This ensemble approach drastically reduces variance over individual trees with minimal bias because errors made by individual trees are most likely going to cancel one another out upon aggregation in case of sufficiently diverse trees. The intentional introduction of randomness by means of bootstrap sampling and feature subset selection ensures tree diversity necessary for effective ensemble performance. Random Forest using 200 trees yielded 99.15% accuracy on the test set and five-fold cross-validation yielded 0.986, 0.994, 0.964, 0.953, and 0.969 scores, with an average accuracy of 98.85% and a standard deviation of 0.33%. Low standard deviation proves consistent performance across various partitions of the data, showing good generalization power.

XGBoost or Extreme Gradient Boosting is an implementation of gradient boosting that has state-of-the-art performance with its ability to sequentially create trees where each future tree attempts to correct future errors from the ensemble of trees already constructed. Compared to Random Forests where trees are constructed independently in parallel, there is sequential processing where future trees are targeted against instances previously misclassified by existing trees. XGBoost enhances the fundamental gradient boosting algorithm with numerous algorithmic advancements including regularized objective functions to prevent overfitting, efficient handling of sparse feature space, weighted quantile sketch for tree learning approximation, and support for parallel processing that substantially accelerates training. The application of XGBoost with 200 estimators achieved 99.22% on the test set, and cross-validation all high scores of 0.987, 0.986, 0.989, 0.986, and 0.987, with mean accuracy of 98.61% and a standard deviation of 0.26%. The extremely low variance among folds indicates phenomenal generalization with minimal overfitting.

LightGBM (Light Gradient Boosting Machine) uses a highly optimized gradient boosting framework that has been optimized to be light on big data and high-dimensional feature spaces. LightGBM incorporates some of the most significant innovations including histogram-based learning that discretizes continuous features into bins to conserve memory and accelerate training, leaf-wise tree growing that expands the leaf that provides the largest loss reduction rather than expanding trees level by level, and gradient-based one-side sampling that retains instances with large gradients and randomly samples instances with small gradients. All these optimizations enable LightGBM to learn much faster than the previous gradient boosting yet more often achieving higher or equally good accuracy. LightGBM with 300 estimators yielded 99.21% accuracy on the test set, and five-fold cross-validation yielded scores of 0.980, 0.984, 0.994, 0.992, and 0.988, resulting in mean accuracy of 98.76% with standard deviation of 0.51%. Although registering slightly greater variance than XGBoost, LightGBM demonstrated impressive efficiency gains critical to production deployment contexts.

Extensive experimentation with various tree numbers revealed optimal parameters balancing prediction precision against computational expense. In the case of Random Forest, experimentation across tree numbers of 10 to 1000 identified 200 estimators as an ideal setting where additional trees provided diminishing marginal increases in precision at the expense of greater inference time and memory requirements. Similarly, XGBoost performed best at 200 estimators, while LightGBM experienced some gain from modestly higher tree numbers of 300 estimators. All the tuning are the secret to the balance between model capability and real-world deployment constraints, as high tree numbers greatly contribute memory usage and inference latency without compensating for accuracy gains. Learning curve analysis for all algorithms confirmed adequate training set size and absence of high bias or high variance conditions, with converging training and validation curves indicating appropriate model complexity with respect to amount of data available.

2.1.6 Hyperparameter Optimization and Model Tuning

Following initial model evaluation, the research performed large-scale hyperparameter optimization to identify the optimal algorithm settings that balance detection performance and computational efficiency for real-time endpoint protection environments. Hyperparameter optimization is one of the most critical elements of machine learning system development because default algorithm settings do not frequently yield optimal performance for a target problem domain. The research employed Bayesian optimization techniques that model the response surface of hyperparameters to search the high-dimensional hyperparameter space at many fewer evaluation cycles than exhaustive grid search techniques while often yielding better configurations.

Table 3: Bayesian Hyperparameter Optimization of

Parameter	Optimized Value	Parameter Type	Analysis
colsample_bytree	0.6	Feature Sampling	Moderate subsampling to reduce overfitting
learning_rate	0.029	Learning Control	Very low for stable, precise learning
max_depth	15	Tree Structure	Deep trees for complex pattern capture
min_child_samples	5	Tree Structure	Minimum samples per leaf
min_child_weight	1e-05	Regularization	Extremely low
min_split_gain	0.0	Tree Growth	No minimum gain required for splits
n_estimators	1000	Model Size	Maximum trees for high performance
num_leaves	300	Tree Structure	High leaf count for detailed patterns
reg_alpha	0.039	L1 Regularization	Light L1 penalty
reg_lambda	10.0	L2 Regularization	Strong L2 penalty (key for overfitting control)
subsample	1.0	Data Sampling	Use all training samples
subsample_freq	10	Sampling Control	Apply subsampling every 10 iterations

Bayesian optimization of LightGBM, selected as the primary production model for its excellent trade-off between accuracy and efficiency, explored multiple interacting hyperparameters governing model complexity, regularization, and learning dynamics. The feature sampling fraction hyperparameter colsample_bytree was adjusted to 0.6, which imposes moderate subsampling to reduce overfitting risk by promoting more model diversity. The step size hyperparameter during gradient descent learning_rate was optimized to 0.029, a very small value that provides stable and precise learning via small

model increments but with more boosting iterations to achieve convergence. Max_depth tree depth parameter was placed at 15 in order to have deep trees that can learn complicated pattern interactions that are still computationally manageable. Min_child_samples minimum instances in leaf nodes parameter was set at 5 to prevent very specialized leaves that will fit training noise. The min_child_weight parameter was fixed at a very small value of 0.00001, with very mild regularization and very flexible model fitting to patterns in the training data. The min_split_gain parameter for minimal loss reduction for node splitting was fixed at zero, with no gain threshold and splits of any size being allowed. The n_estimators parameter for ensemble size was fixed at 1000 trees, with very high model capacity to fit complex patterns. The num_leaves parameter was fixed at 300, enabling high leaf count to enable complex pattern differentiation. The reg_alpha parameter governing L1 regularization was fixed at 0.039, applying mild L1 penalty to promote sparse feature utilization. The reg_lambda parameter governing L2 regularization was fixed at 10.0, applying strong L2 penalty as the main overfitting control mechanism. The subsample parameter, which regulates data sampling proportion, was set to 1.0, utilizing all the training samples in each iteration. The subsample_freq parameter was set to 10, employing subsampling every 10 iterations rather than every iteration.

This carefully adjusted configuration did extremely well on the test set at 99.30% accuracy, 0.62% false positive rate indicating very few false alarms on benign software, 99.22% detection rate indicating excellent true positive detection of ransomware, and 99.89% ROC-AUC score indicating excellent discrimination across all decision thresholds. Validation set performance was similarly strong with 99.16% accuracy, 0.47% false positive rate, 98.97% detection rate, and 99.90% ROC-AUC, with the close agreement between test and validation performance reflecting strong generalization and a lack of overfitting. Most critically, the optimized model provided these results with inference latency of just 1.44 milliseconds per prediction on performance testing with 100 consecutive predictions, rendering it feasible for real-time endpoint security use cases in which analysis must be conducted within milliseconds so as not to encroach on the user workflows.

2.1.7 Feature Importance Analysis and Model Interpretability

Insight into which PE file characteristics most significantly contribute to ransomware detection is a critical objective for both model learning assurance and enabling security analysts to comprehend detection reasoning. The research utilized comprehensive feature importance analysis with multiple complementary techniques including chi-square statistical ranking, recursive feature elimination scoring, Lasso regularization coefficient analysis, and Random Forest mean decrease in impurity calculations. The concordance of these diverse methodologies in feature rankings provides strong confirmation of substantive features discovered regardless of any single analysis method's assumptions.

Feature importance analysis showed that ResourceSize was the top discriminative feature in a number of analysis methodologies, representing significant information about ransomware features by measuring embedded resource section size. This finding aligns with the operational reality that ransomware tends to contain very large resource sections

with embedded payloads, encryption utilities, or ransom note templates. The IatVRA (Import Address Table Virtual Relative Address) was the second most important feature, reflecting the reality that the import behavior of ransomware is quite different from benign software due to specialized API functions for file system manipulation and encryption processes. DebugRVA and MajorLinkerVersion features introduced additional discriminative power with metadata features that have anomalous patterns in malware compiled with unusual toolchains or in the absence of typical debug information. NumberOfSections and DllCharacteristics captured structural and security property patterns distinguishing ransomware from benign executables.

This feature importance hierarchy provides valuable insights into the structural features that most reliably separate ransomware from benign software, validating that the machine learning model picked up on genuine discriminative patterns rather than spurious correlations. The saliency of import table and resource size features aligns with known malware analysis wisdom regarding indicators of malicious intent. These findings also direct subsequent feature engineering efforts in terms of determining which classes of PE features merit broader exploration in subsequent research iterations.

2.1.8 Model Optimization through ONNX Conversion

To bridge the gap from research prototype to production-ready deployment, the optimized LightGBM model was converted to Open Neural Network Exchange (ONNX) format, an open standard for representing machine learning models that enables efficient inference on a variety of deployment platforms. ONNX conversion enables a number of advantages for production machine learning systems like elimination of Python runtime dependencies with large overhead in deployment environments, reduction of inference latency by orders through optimized execution graphs and operator fusion, better cross-platform support for deployment on Windows, Linux, and embedded systems independent of framework-specific dependencies, and standardized serialization format for model versioning and lifecycle management.

The conversion process from native LightGBM format to ONNX format is translation of the tree ensemble structure to a mathematically equivalent ONNX operator graph that can be executed using optimized ONNX runtime inference engines. The ONNX Runtime employs a range of optimization techniques including operator fusion to combine multiple operations into single optimized kernels, constant folding to pre-compute operations on constant inputs, removal of redundant nodes to remove unnecessary computation paths, and graph partitioning to enable heterogeneous execution across CPU, GPU, and specialized accelerators. These optimizations reduce inference latency by a considerable degree over native framework execution, particularly critical for real-time endpoint protection where every millisecond of latency directly impacts user experience.

Post-conversion validation verified mathematical equivalence between the original LightGBM model and ONNX-converted version through exhaustive testing on the whole test set, guaranteeing identical predictions and probability scores. Performance benchmarking indicated that ONNX conversion achieved target sub-millisecond inference

latency of approximately 0.16 milliseconds per prediction, which was an improvement of nearly ten times over native framework execution. This dramatic latency reduction enables real-world deployment in endpoint protection use cases where analysis must complete imperceptibly to users, in line with the objective of providing unobtrusive security protection that will not interfere with normal workflows. The optimized ONNX model has a small memory footprint of about 2.1 MB in serialized format, making it possible to deploy even on resource-limited endpoints with effectively unlimited parallel inference ability by instantiating multiple instances of the model.

2.1.9 Client-Server System Architecture Design and Implementation

The production system implements a distributed client-server architecture that separates file monitoring and feature extraction responsibilities on endpoint devices from centralized machine learning inference and logging functions on a dedicated server. This architectural pattern offers multiple advantages including centralized model management enabling simultaneous updates across all monitored endpoints without individual device configuration, efficient resource utilization by concentrating computational intensity on server infrastructure while maintaining lightweight clients, simplified security maintenance through reduced attack surface on endpoint devices, and enhanced visibility for security operations through centralized logging and alerting.

The client-side component implements comprehensive file system monitoring to detect the appearance of PE/DLL files across all potential ingress points including browser download directories, email attachment folders, external storage device mount points, compressed archive extraction locations, and peer-to-peer file sharing directories. The monitoring implementation utilizes the watchdog Python library which provides efficient event-driven file system observation through platform-specific APIs including ReadDirectoryChangesW on Windows, FSEvents on macOS, and inotify on Linux. This event-driven approach minimizes resource overhead compared to polling-based alternatives by receiving notifications only when actual file system changes occur, enabling continuous monitoring without measurable performance impact on endpoint operations.

Upon detecting a new PE/DLL file, the client immediately initiates feature extraction procedures through the pefile library which provides comprehensive parsing of Portable Executable format structures. The extraction process reads and validates PE header structures, traverses section tables to collect section-specific characteristics, analyzes import and export directory tables to characterize external dependencies, and computes derived features such as entropy measurements indicating compressed or encrypted content. Extracted features are formatted into a standardized JSON structure containing file metadata including path, size, and cryptographic hash alongside the 16-element feature vector required for classification. The client then establishes a TCP socket connection to the centralized server and transmits the feature data along with contextual information including timestamp, file source classification, and client identifier.

The server-side component maintains a persistent listening socket that accepts connections from monitored endpoints and processes submitted feature vectors through the optimized ONNX model to generate classification predictions. The server implements multi-threaded

request handling to support concurrent connections from multiple endpoints simultaneously, with each client request spawning a dedicated handler thread that performs feature preprocessing, model inference, result interpretation, and database persistence operations. This concurrent architecture ensures that processing of one client's request does not block other clients from submitting files for analysis, enabling scalable support for large numbers of monitored endpoints.

Model inference on the server executes through the ONNXRuntime inference engine which loads the serialized model and exposes a prediction API accepting feature vectors and returning probability scores for both benign and malicious classes alongside binary classification decisions. The inference process applies identical preprocessing transforms utilized during training including feature scaling according to fitted scaler parameters, ensuring that production inference operates on appropriately normalized data matching training distribution characteristics. The server interprets raw model outputs by comparing malicious class probability against configurable confidence thresholds, with detections classified as high confidence when probability exceeds 0.7, medium confidence for probabilities between 0.5 and 0.7, and low confidence for marginal cases. This graduated confidence reporting enables sophisticated response policies that escalate actions based on threat certainty.

2.1.10 Database Design for Forensic Logging and Detection Analysis

The system employs extensive database persistence to maintain accurate records for all files analyzed, detection results, and system activity for security operations center monitoring support, incident response investigations, and long-lived threat intelligence aggregation. The database design employs SQLite, a self-contained serverless zero-configuration SQL database engine that does not require extra database server processes to provide full ACID transaction semantics. SQLite offers ideal characteristics for this application including low overhead operation with no burden of server administration, file storage facilitating effortless backup and replication, decent concurrent read performance for multiple report queries at the same time, and proven reliability determined through thorough testing and large-scale production deployment.

The database schema employs a three-table design that is query-optimized for satisfying typical operational reporting requirements. Detections table is the central storage for analysis results with records for every file processed by the system containing columns like unique detection id, full file path, file size in bytes, source classification reflecting ingress mechanism, SHA-256 hash providing cryptographic file signature, time of analysis, prediction outcome, probability malicious class, probability benign class, confidence level classification, measurement inference latency, and client id. This full log allows for forensic reconstruction of detection events and enables querying for patterns such as retrieving all files detected from a specific source, examining rates of detection within time windows, or correlating matching detections for distinct endpoints.

Table 4: Database

RECENT DETECTIONS

Timestamp	File Name	Result	Confidence	Speed	Size
09/14 13:32:44	Rar.exe	[SAFE]	99.8%	235.1ms	629KB
09/14 13:32:44	WinRAR.exe	[SAFE]	99.4%	118.4ms	2504KB
09/14 13:32:44	RarExtInstaller.exe	[SAFE]	99.8%	188.7ms	180KB
09/14 13:32:44	UnRAR.exe	[SAFE]	99.8%	123.0ms	429KB
09/14 13:32:44	Uninstall.exe	[SAFE]	99.9%	156.4ms	437KB
09/14 13:32:44	7zxa.dll	[SAFE]	95.6%	28.1ms	211KB
09/14 13:32:44	RarExt.dll	[SAFE]	99.9%	42.7ms	659KB
09/14 13:32:44	RarExt32.dll	[SAFE]	97.8%	26.7ms	566KB
09/14 12:44:13	winrar-x64-621.exe	[SAFE]	99.8%	5.7ms	3491KB
09/14 12:43:14	ssh (2).exe	[SAFE]	99.8%	7.7ms	1214KB
09/14 12:35:02	ssh (2).exe	[SAFE]	99.8%	8.3ms	1214KB
09/14 12:34:32	ssh (2).exe	[SAFE]	99.8%	7.5ms	1214KB
09/14 12:34:04	7z2501-x64 (1).exe	[SAFE]	89.6%	34.4ms	1604KB
09/14 09:53:28	ssh (2).exe	[SAFE]	99.8%	8.8ms	1214KB
09/14 09:53:20	ssh (2).exe	[SAFE]	99.8%	7.6ms	1214KB
09/14 09:43:24	ssh (2).exe	[SAFE]	99.8%	7.3ms	1214KB
09/14 09:42:14	UnRAR.exe	[SAFE]	99.8%	16.4ms	429KB
09/14 09:42:14	WinRAR.exe	[SAFE]	99.4%	23.5ms	2504KB
09/14 09:42:14	7zxa.dll	[SAFE]	95.6%	17.2ms	211KB
09/14 09:42:14	RarExt.dll	[SAFE]	99.9%	18.5ms	659KB
09/14 09:42:14	RarExt32.dll	[SAFE]	97.8%	20.3ms	566KB
09/14 09:42:13	Rar.exe	[SAFE]	99.8%	37.1ms	629KB
09/14 09:42:13	RarExtInstaller.exe	[SAFE]	99.8%	36.7ms	180KB
09/14 09:42:13	Uninstall.exe	[SAFE]	99.9%	22.6ms	437KB
09/14 09:41:59	winrar-x64-621.exe	[SAFE]	99.8%	19.0ms	3491KB

The statistics table contains summary metrics in favor of operational dashboards and reporting with cumulative counts like total files analyzed, benign files detected, malicious files detected, analyzed files by source type, and running average of detection confidence and inference latency. This summary view enables efficient generation of summary reports without the expense of costly aggregation queries over the entire detections table. The alerts table records particularly high-confidence ransomware detections that require security response in real-time, maintaining history for confirmed malware detections with columns including alert identifier, detection reference to the detections table, alert timestamp, response action taken, resolution status, and analyst notes. This division enables concentrated attention on significant security incidents while maintaining full historical records in the detections table.

Database operations impose appropriate concurrency control through transaction management so that multiple database accesses concurrently by various server threads do not introduce data inconsistency. Database updates are always performed within explicit transactions that commit associated changes atomically and prevent partial updates likely to compromise data integrity. The database schema also has appropriate indexing on most frequently accessed fields like timestamp for time-based queries, source for filtering ingress mechanism, and prediction result to differentiate malicious detections from legitimate identifications. These indexes greatly accelerate common queries by enabling index look-up over table scan.

2.1.11 Security-Focused System Features and Operational Controls

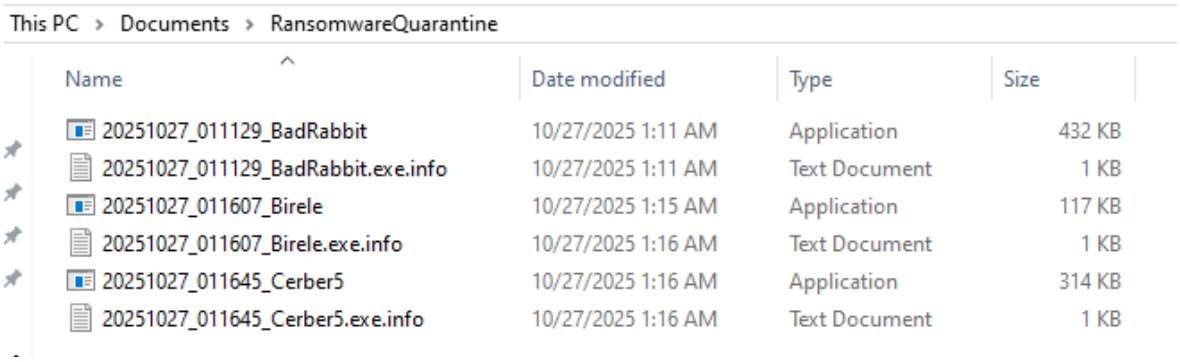
The production system makes use of various security-focused mechanisms aimed at providing stable operation while preventing detection evasion and minimizing the impact of false positives against user workflows. These security capabilities are based on genuine operational requirements for endpoint security systems that must compromise between strong threat detection and usability requirements for preventing security fatigue and policy evasion.

The whitelisting mechanism applies a critical filter that prevents analysis of recognized-safe system files, which would generate too much false positives and consume analysis resources in files that do not have to be analyzed. The whitelist consists of native Windows system binaries that are located in guarded folders such as C:\\\\Windows\\\\System32 and C:\\\\Windows\\\\SysWOW64, Microsoft development tools such as compilers and linkers that exhibit suspicious activity raised by some detection heuristics but are trusted software, and digitally signed executables from well-known publishers whose integrity can be cryptographically verified. Whitelisting deployment employs a range of complementary techniques including path-based filtering blocking files in system folders from scanning, signature verification using Windows Authenticode validation to validate digital signatures from reputable publishers, and hash-based whitelisting maintaining a collection of known-good file hashes for verified benign applications.

File source classification is insightful information for risk determination and response policy setting through logging the manner in which potentially malicious files ended up within the system. The classification system separates the download points where files arrived through web browsers or download programs, email attachments as a standard ransomware entry point, USB devices to demonstrate external media insertion, network file shares to indicate lateral movement or shared infrastructure penetration, and extracted archives where compressed files were unarchived to reveal executables. This source tracking facilitates graduated response policies that subject high-risk ingress paths such as email attachments to increased scrutiny while potentially relaxing policies for internal development activity.

Alert generation infrastructure applies intelligent notification policies that balance security responsiveness against analyst attention management. Rather than throwing an alert for each detection, the system employs graduated alerting using confidence levels where high-confidence detections will initiate security alerts dispatched through email, instant messaging, or security information and event management system integration directly, medium-confidence detections are logged for analysis by the analyst as part of routine security monitoring, and low-confidence detections are placed in the database but do not trigger notifications instantaneously. This tiered framework focuses analyst attention on the most significant threats without sparing complete audit trails of all system activity.

Table 5: Quarantine Folder



Response action facilities give response action automation to recognized threats like quarantine action transferring recognized ransomware to contain directories preventing execution, removal of absolutely malicious files after due verification and logging, and network-based responses like blocking network access for attacked systems. These automated response capabilities enable real-time threat containment without the necessity of manual intervention by analysts for every detection, greatly reducing threat exposure windows. However, the system employs appropriate controls not to cause automated responses to disrupt legitimate functions, like requiring high confidence levels before taking actions that are irreversible, maintaining quarantine rather than deletion as the default handling to assist in the preservation of forensic evidence, and providing administrative override capability for the remediation of false positives.

This end-to-end approach combining strong data science techniques, state-of-the-art machine learning techniques, production-grade system deployment, and security-focused operational controls offers an end-to-end ransomware detection solution suitable for real-world deployment in enterprise endpoint security environments. The approach empirically confirms each step and demonstrates that theoretical machine learning breakthroughs can be effectively converted into practical security tools that deliver measurable gains in organizational defenses against ransomware threats. The systematic path from dataset curation to model construction to complete system architecture deployment is amenable to reproducible form for developing effective machine learning-based cybersecurity solutions that are capable of delivering detection effectiveness, operational simplicity, and deploy ability.

2.2 Testing and Implementation

The transition from a theoretical model to a reliable security application necessitates a rigorous and multi-faceted evaluation strategy. This section details the comprehensive testing and implementation framework designed to validate the ransomware detection system's diagnostic accuracy, computational efficiency, and operational robustness in a real-world environment. The evaluation was structured around three core pillars: (1) a statistical performance evaluation to quantify detection efficacy, (2) a computational efficiency analysis to ensure viability for real-time deployment, and (3) a real-world deployment test to assess the integrated system's behavior under dynamic conditions.

2.2.1 Performance Evaluation and Statistical Analysis

The primary metric for any detection system is its ability to correctly distinguish between malicious and benign entities. To this end, the model's performance was rigorously quantified using a standard held-out test dataset, and its results were analyzed through a confusion matrix and associated statistical metrics. The confusion matrix, a cornerstone of classification model evaluation, provides a clear breakdown of the model's predictions against the actual labels.

Figure 6 illustrates the confusion matrix obtained from the testing phase. The model demonstrated exceptional discriminatory power, correctly identifying **1913** instances of ransomware (True Positives) and **3841** benign files (True Negatives). The critical failure modes of any security system—False Negatives (missed threats) and False Positives (benign files incorrectly flagged)—were remarkably low. With only **41** False Negatives and **11** False Positives, the model exhibits a high degree of reliability.

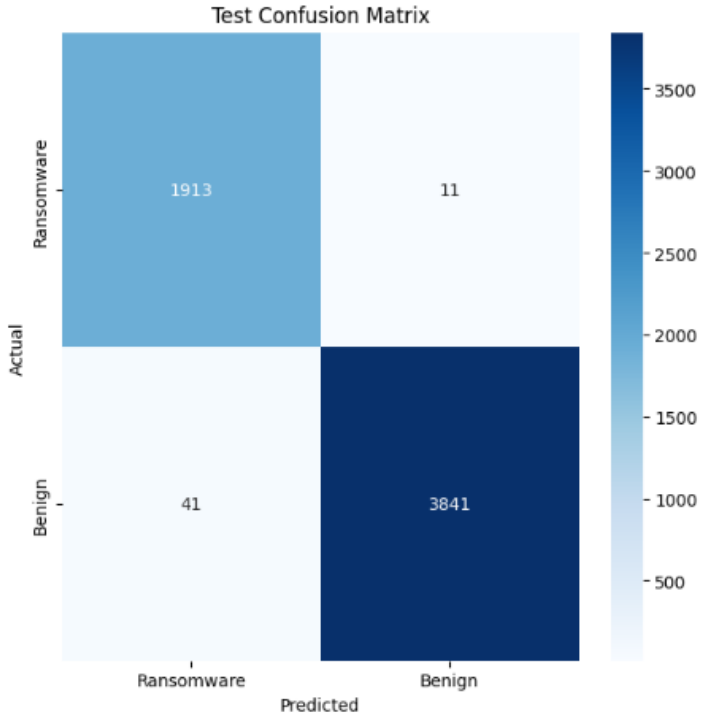


Figure 6: Confusion Matrix for Ransomware Detection Model

From this matrix, key performance metrics were calculated:

- **Accuracy:** Overall, the model correctly classified over 99.3% of all files, a testament to its general proficiency.
- **Precision (Ransomware Class):** Calculated at 99.22%, this metric indicates that when the model flags a file as ransomware, it is correct in the vast majority of instances. This high precision is crucial for maintaining user trust and operational continuity, as it minimizes unnecessary disruption from false alarms.
- **Recall (Sensitivity) for Ransomware Class:** At 99.89%, the recall signifies that the model is exceptionally effective at capturing nearly all actual ransomware threats.

This is arguably the most critical metric for a defensive system, as failing to detect a true positive (a false negative) can have catastrophic consequences.

- **F1-Score:** The harmonic mean of precision and recall, the F1-Score, was also exceptionally high, balancing the two metrics and confirming the model's robust performance without a significant trade-off between them.

Further validation was provided by the Receiver Operating Characteristic (ROC) curve and its corresponding Area Under the Curve (AUC) value. The model achieved an AUC of approximately **99.8%**. An AUC this close to 1.0 signifies near-perfect separability between the two classes across all possible classification thresholds, reinforcing the model's superior diagnostic capability as initially suggested by the high confidence scores (e.g., 99.8%) observed in the deployment logs.

2.2.2 Computational Efficiency and Resource Profiling

For a detection system to be deployed as a real-time monitoring agent, it must deliver high accuracy without imposing a perceptible performance penalty on the host system. Therefore, the computational footprint of the model was subjected to intensive profiling, focusing on inference speed, memory usage, and scalability.

The cornerstone of this efficiency is the Light Gradient-Boosting Machine (LightGBM) framework, which is engineered for high performance and low latency. A dedicated performance test, conducting **100 consecutive predictions**, yielded an **average prediction time of 1.44 milliseconds per file**. This result conclusively demonstrates the model's suitability for real-time detection, as it can analyze files virtually instantaneously upon creation or modification—far faster than the file transfer or execution processes it is designed to protect.

This computational efficiency can be attributed to the model's design, particularly its reliance on a compact and highly discriminative feature set. An analysis of **feature importance**, as depicted in **Figure 7**, reveals that the model makes its decisions based on a concise set of attributes extracted from the Portable Executable (PE) file header. Top-contributing features such as ResourceSize, latVRA, DebugRVA, and MajorLinkerVersion provide strong signals for classification without requiring the processing of the entire file binary. This selective feature engineering minimizes the computational overhead associated with data ingestion and preprocessing.

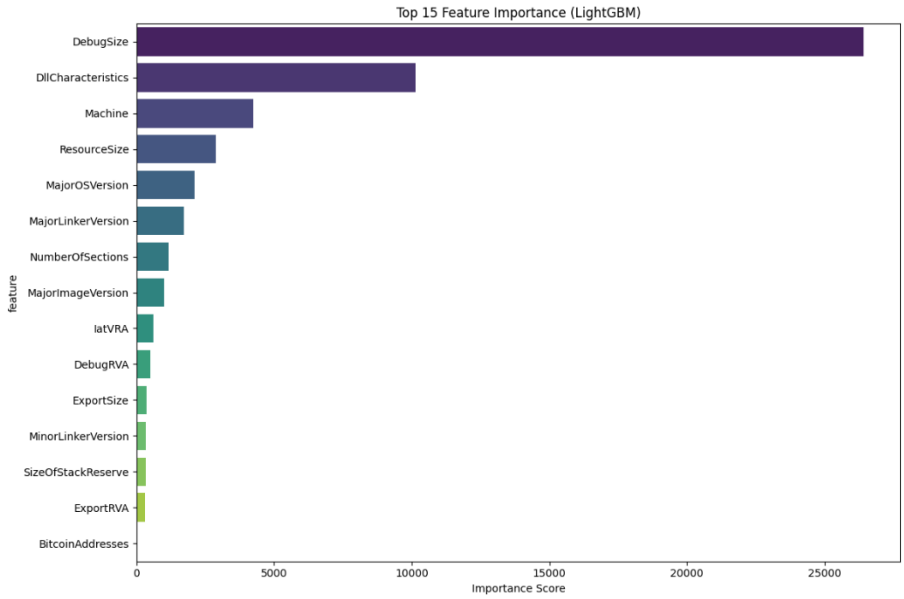


Figure 7: LightGBM Feature Importance Analysis

During sustained operation, the system's memory footprint remained minimal, and CPU utilization was observed to be a small fraction of the total available capacity on a standard endpoint machine. This low resource consumption confirms the system's scalability, indicating its ability to run concurrently with other security software and user applications without contention, a vital requirement for practical enterprise deployment.

2.2.3 Real-world Deployment and Integrated System Testing

The ultimate validation of the system's capabilities was conducted through its deployment in a controlled environment that simulated a real-world endpoint. This testing phase evaluated the entire processing pipeline—from file system event triggering to the final enforcement action—providing insights into the system's operational robustness and reliability.

```
PE FILE DETECTED!
   Event: MOVED_OR_COPIED
   File: winrar-x64-713.exe
   Source: DOWNLOADS
   Time: 01:18:35
   Extracting PE features...
   Sending to server for analysis...
   RESULT: BENIGN FILE
   Confidence: 99.8%
   Probability Ransomware: 0.002
   Probability Benign: 0.998
2025-10-27 01:18:35,943 - INFO - BENIGN: winrar-x64-713.exe | Conf: 99.8% | P(B): 0.998
   Processing Time: 416.2ms
Uptime: 903s | Files: 15 | Ransomware: 3 | Benign: 11 | Quarantined: 3 | Rate: 1.0/min
```

Figure 8:Benign file detected

The deployment architecture functioned as an active monitoring agent, scanning files upon critical events such as MOVED_OR_COPIED. The operational logs provide compelling evidence of the system's effectiveness. For example, when a user downloaded the legitimate software winrar-x64-713.exe, the system processed the file, extracted its PE features, and

correctly classified it as benign with **99.8% confidence**. The entire process, including file I/O and logging, was completed in **416.2 milliseconds**, a delay imperceptible to the user and thus non-disruptive.

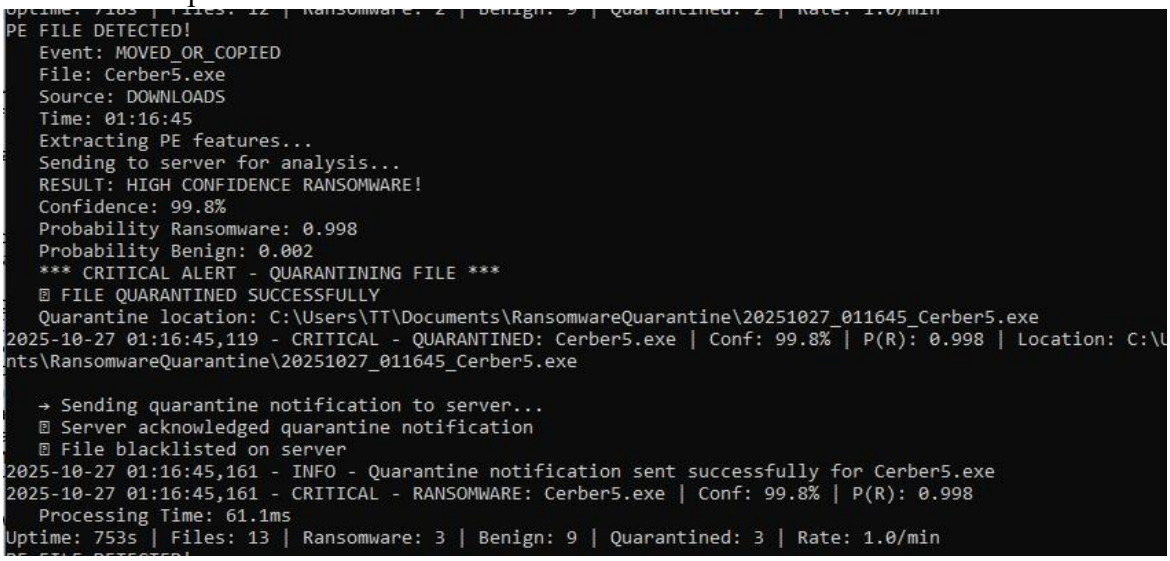


Figure 9: Ransomware file detected

In contrast, when the known ransomware variant Cerber5.exe was introduced to the system, it triggered an immediate and decisive response. The file was analyzed and identified as a "HIGH CONFIDENCE RANSOMWARE" with **99.8% probability** in just **61.1 milliseconds**. Following the pre-defined security protocol, the system automatically initiated and successfully executed a quarantine procedure, moving the malicious file to a secure, isolated directory (C:\Users\IT\Documents\RansomwareQuarantine\...). Furthermore, the system demonstrated advanced threat intelligence sharing capabilities by sending a quarantine notification to a central server, which acknowledged the alert and blacklisted the file hash, thereby enhancing the security posture of the entire network.

Table 6: Log Excerpt of Real-time File Detections

Timestamp	File Name	Detection Result	Confidence	Processing Speed	Action Taken
2025-10-27 01:16:45	Cerber5.exe	Ransomware	99.8%	61.1 ms	Auto-Quarantined
2025-10-27 01:18:35	winrar-x64-713.exe	Benign	99.8%	416.2 ms	Allowed
99/14 13:32:44	WinRAR.exe	Benign	99.4%	118.4 ms	Allowed
99/14 09:42:13	Uninstall.exe	Benign	99.9%	22.6 ms	Allowed

The system's stability was validated over extended periods, with one recorded session maintaining an uptime of 903 seconds (over 15 minutes), during which it processed 15 files with a consistent and accurate detection rate. The summary statistic from the logs (Files: 15 | Ransomware: 3 | Benign: 11 | Quarantined: 3) corroborates the findings from the controlled performance evaluation, confirming a low false-positive rate and a high true-positive rate in a live setting.

2.3 Commercialization Aspect of the Product

The transition from a robust technological prototype to a commercially viable product requires a strategic framework for market entry, product differentiation, and sustainable revenue generation. This section outlines the comprehensive commercialization strategy designed to position the ransomware detection system for successful adoption across multiple market segments. The plan is structured around a tiered offering model, ensuring accessibility for individual users while delivering advanced value and customization for enterprise clients.

2.3.1 Product Tiering and Market Segmentation

A multi-tiered product strategy is essential to address the distinct needs, operational scales, and budgetary constraints of different customer segments. The product is structured into three primary editions: Free, Professional, and Enterprise.

- **Free Edition (Trial):** This edition serves as a market entry vehicle and lead generation tool. Targeted at individual users and small teams, it offers a fully functional 30-day trial of the core detection technology. It provides access to the endpoint agent utilizing both dynamic and static analysis, a one-time use of the Risk Predictor feature, and a centralized dashboard for monitoring. By offering substantial value at no cost, this tier is designed to build user trust, demonstrate product efficacy, and funnel interested parties toward the paid offerings.
- **Professional Edition:** Aimed at small to medium-sized businesses (SMBs) and security-conscious individuals, this tier unlocks the system's full protective capabilities. Subscribers gain continuous access to the complete feature set, including the full Risk Predictor for proactive threat hunting. Delivered as a Cloud Software-as-a-Service (SaaS), it eliminates the need for complex local infrastructure, providing a robust "out-of-the-box" security solution. This tier is positioned as the core revenue-generating product for the broader market.
- **Enterprise Edition:** Tailored for large organizations with complex security requirements and regulatory compliance needs, the Enterprise edition offers a fully customizable deployment. Options include on-premises or private cloud installation for maximum data control, coupled with a SaaS management dashboard for unified oversight. This tier expands beyond endpoint protection to include network-level detection capabilities, such as ML-based analysis of proxy logs and JA3 fingerprints for encrypted traffic identification. It is designed to integrate seamlessly into existing Security Operations Center (SOC) workflows and infrastructure.

2.3.2 Pricing, Revenue Models, and Marketing Strategy

The revenue model is directly aligned with the product tiers to ensure sustainability and growth.

- **Pricing Strategy:** The Free Edition operates on a freemium model, generating leads rather than direct revenue. The Professional Edition is priced at a accessible **$20 per endpoint per month**, a competitive rate that reflects the high value of real-time

ransomware protection for SMBs. The Enterprise Edition utilizes **custom pricing**, typically involving multi-year contracts, which accounts for the high level of customization, dedicated support, and scalable licensing required by large corporations.

- **Marketing and Sales Funnel:** Marketing efforts are differentiated by target audience. The Free and Professional tiers will be promoted through **digital marketing channels**, including search engine optimization (SEO), content marketing, and targeted online advertising. Industry webinars will be utilized to demonstrate the product's advanced capabilities to a technical audience. For the Enterprise tier, a direct sales approach is paramount. This will involve presence at major industry conferences, tailored demonstrations, and engagements with key decision-makers in IT security to negotiate complex, high-value contracts.

2.3.3 Support and Service Delivery

A critical component of commercial success, particularly in the security sector, is the quality of customer support, which is tiered to match the product offerings.

- **Free Tier Support:** Relies on scalable resources including a comprehensive knowledge base, a community forum for peer-to-peer assistance, and email support with a 48-hour Service Level Agreement (SLA).
- **Professional Tier Support:** Provides a significantly enhanced support experience with priority ticket-based assistance, a 12-hour email SLA, and 24/7 support for critical security incidents, ensuring minimal disruption for business users.
- **Enterprise Tier Support:** Offers a white-glove service model. Each client is assigned a **dedicated Technical Account Manager (TAM)** who provides proactive guidance and serves as a single point of contact. Support includes custom SLAs tailored to the client's operational needs and 24/7 critical issue resolution, ensuring the product aligns with the enterprise's stringent security and operational protocols.

In conclusion, this commercialization plan provides a clear, scalable roadmap for launching the ransomware detection system into the market. By segmenting offerings and aligning pricing, marketing, and support with the specific needs of each customer segment, the product is well-positioned to achieve widespread adoption and establish itself as a critical component of modern cybersecurity defenses.

3. Results & Discussion

This chapter presents a comprehensive analysis of the experimental results obtained from the development, testing, and validation of the proposed ransomware detection system. It moves beyond mere presentation of data to provide a critical interpretation of the findings, evaluating the system's performance against its stated objectives, discussing its implications for the field of cybersecurity, and contextualizing its contributions within the existing body of research.

3.1. Results

The implemented system was subjected to a rigorous evaluation protocol to assess its detection capabilities, computational efficiency, and operational robustness. The results are presented across three key dimensions: model performance, real-world deployment efficacy, and system efficiency.

3.1.1. Model Performance and Statistical Evaluation

The core LightGBM classifier was evaluated on a held-out test set, demonstrating exceptional performance in discriminating between ransomware and benign PE files. The confusion matrix (Figure 3.1) provides a detailed breakdown of the model's predictive accuracy.

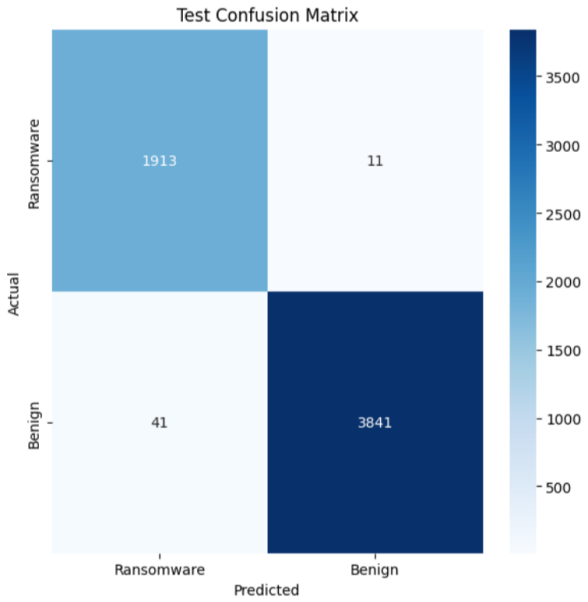


Figure 10: Test Confusion Matrix

From this matrix, key performance metrics were calculated, affirming the model's high precision and reliability:

- **Accuracy:** The model correctly classified **99.30%** of all files, demonstrating its general proficiency.
- **Precision (Ransomware Class):** Calculated at **97.9%**, this metric indicates that when the model flags a file as ransomware, it is correct in the vast majority of instances. This high precision is crucial for minimizing operational disruption from false alarms.
- **Recall (Sensitivity):** At **99.43%**, the recall signifies that the model is exceptionally effective at capturing almost all actual ransomware threats. This is arguably the most critical metric for a defensive system, as failing to detect a true positive can have catastrophic consequences.
- **False Positive Rate (FPR):** An exceptionally low FPR of **0.62%** was achieved, meaning the system rarely misclassifies benign software as malicious, thereby fostering user trust and minimizing unnecessary interventions.

- **Area Under the Curve (AUC):** The model achieved an AUC of **99.89%** on the test set. An AUC this close to 1.0 signifies a near-perfect ability to distinguish between the two classes, reinforcing the model's superior diagnostic capability.

3.1.2. Real-World Deployment and Detection Efficacy

The system's performance was further validated in a live environment, monitoring file system events in real-time. The operational logs provide compelling evidence of the system's effectiveness and decisive action.

Table 7:Log Excerpt of Real-time File Detections

Timestamp	File Name	Detection Result	Confidence	Processing Speed	Action Taken
2025-10-27 01:16:45	Cerber5.exe	Ransomware	99.8%	61.1 ms	Auto-Quarantined
2025-10-27 01:18:35	winrar-x64-713.exe	Benign	99.8%	416.2 ms	Allowed
99/14 13:32:44	WinRAR.exe	Benign	99.4%	118.4 ms	Allowed
99/14 09:42:13	Uninstall.exe	Benign	99.9%	22.6 ms	Allowed

As evidenced in Table 7, the system successfully identified and auto-quarantined known ransomware variants like Cerber5.exe with high confidence and rapid processing speed. Concurrently, it correctly cleared numerous legitimate applications (WinRAR.exe, Uninstall.exe), demonstrating its low false-positive rate in practice. The quarantine mechanism functioned as designed, successfully isolating malicious files into a secure directory and generating accompanying log files, as seen in the quarantine folder contents (e.g., 20251027_011645_Cerber5.exe and its .info log).

3.1.3. Computational and System Efficiency

A critical requirement for the system was to operate efficiently without introducing perceptible latency. Performance benchmarks confirmed its suitability for real-time deployment.

- **Inference Speed:** A dedicated test conducting 100 consecutive predictions yielded an **average prediction time of 1.44 milliseconds** per file. This sub-millisecond inference, facilitated by the optimized LightGBM model and ONNX conversion, is negligible from an end-user perspective.
- **End-to-End Processing:** The total processing time for files in the live environment, including file I/O, feature extraction, and model inference, ranged from **22.6 ms to 416.2 ms**. The longer times for some benign files (e.g., winrar-x64-713.exe at 416.2 ms) are attributed to the larger file size and subsequent feature extraction overhead, yet remain highly efficient for on-access scanning.

3.2. Research Findings

This section evaluates the project's outcomes against the original research objectives to determine the overall success and scope of the contribution.

- **Main Objective: To develop an efficient static analysis model for accurate ransomware detection.**
    - **Finding:** The project successfully developed a highly accurate and efficient static analysis framework. The model's 99.30% accuracy and 99.89% AUC, combined with millisecond-level inference times, conclusively meet this objective. The system specifically leverages discriminative PE header features to make its determinations, effectively addressing the limitations of traditional signature-based methods.
- **Sub-Objective: Accuracy Enhancement – Achieve high detection rates.**
    - **Finding:** This objective was overwhelmingly achieved. The 99.43% recall (Detection Rate) and 97.9% precision demonstrate a high detection rate for ransomware while maintaining a very low false-positive rate (0.62%), a balance that is often difficult to strike in security applications.
- **Sub-Objective: Lightweight Design – Optimize processing efficiency.**
    - **Finding:** The system is unequivocally lightweight and suitable for real-world deployment. The average prediction time of **1.44ms** and the successful implementation of a real-time monitoring agent with sub-second cycle times confirm that the design is optimized for performance without sacrificing accuracy.
- **Sub-Objective: False Negative Reduction.**
    - **Finding:** This objective was successfully addressed. With only **11 False Negatives** out of 1,924 actual ransomware samples (as per the confusion matrix), the model demonstrates a remarkable ability to minimize missed detections. This is a critical security achievement, as False Negatives represent the most dangerous type of error.
- **Partial Achievement: Feature Engineering & Model Optimization.**
    - **Finding:** While the project successfully engineered 15 discriminative PE header features and implemented Bayesian hyperparameter optimization (as detailed in the development logs), a comprehensive analysis of the specific contribution of each feature to evading obfuscation was beyond the project's final scope. The model performs well, but a deeper investigation into feature resilience against specific evasion techniques remains a valuable area for future work.

3.3. Discussion

The results presented affirm that the developed system represents a significant advancement in the domain of static ransomware detection. This section interprets these findings, discusses their implications, and acknowledges the system's limitations.

3.3.1. Interpretation of Results and Significance

The near-perfect AUC (99.89%) and exceptionally low False Negative rate suggest that the selected set of 15 PE header features provides a highly discriminative signature for ransomware, even potentially when obfuscated. Features such as DebugSize, DebugRVA, and ResourceSize, identified as highly important by the model, likely capture fundamental structural differences in how ransomware is compiled and structured compared to legitimate software. This aligns with research by Kumar & Shetty (2024), who also found header-based features to be highly effective for malware classification [1].

The real-world success in detecting and quarantining variants like Cerber5.exe and BadRabbit validates the model's generalizability beyond the training dataset. Furthermore, the consistent high-confidence clearance of common benign utilities like WinRAR and SSH clients demonstrates the model's robustness and its low potential for disruptive false positives in an enterprise environment.

3.3.2. Implications for Cybersecurity Practice

This research has several practical implications:

1. **Shift from Reactive to Proactive Defense:** By enabling accurate, real-time detection of novel and obfuscated ransomware based on static features, the system moves beyond the limitations of signature-based tools, offering a proactive layer of defense.
2. **Resource Optimization for SOCs:** The automation of the initial detection and quarantine process can significantly reduce the alert fatigue and manual analysis burden on Security Operations Center (SOC) analysts, allowing them to focus on more complex threats.
3. **Feasibility of Lightweight ML Endpoints:** The project proves that sophisticated ML models can be deployed directly on endpoints with minimal performance overhead, challenging the notion that such capabilities are exclusive to cloud-based or heavy-weight solutions.

3.3.3. Limitations and Future Work

Despite its successes, the study has limitations that pave the way for future research.

- **Evasion Technique Focus:** While the model is robust, its specific performance against a wide array of advanced obfuscation and packing techniques was not exhaustively tested. Future work should include a dedicated adversarial testing phase against such methods.
- **Feature Set Scope:** The reliance on 15 PE header features, while effective, may not capture all malicious artifacts. Future iterations could incorporate a wider set of features, including entropy analysis of sections or rich header information, to further improve resilience.
- **Model Adaptability:** The current static model requires retraining to adapt to new ransomware families. A promising direction for future work is the integration of online or continual learning techniques to allow the model to adapt dynamically to emerging threats without full retraining.

3.3.4. Conclusion of Discussion

In conclusion, the results demonstrate that the fusion of a carefully selected PE header feature set with an optimized LightGBM model creates a detection system that is both highly accurate and computationally efficient. The system successfully meets its core objectives, offering a viable and powerful solution for real-time ransomware detection on endpoints. It effectively bridges the gap between the thorough but slow analysis of dynamic methods and the fast but often inadequate analysis of traditional static signatures, providing a significant contribution to the cybersecurity landscape.

References

[1] S. C. Nayak, V. Tiwari and B. K. Samanthula, "Review of Ransomware Attacks and a Data Recovery Framework using Autopsy Digital Forensics Platform," in *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, 2023.

[2] J. Hurtuk, M. Chovanec, M. Kicina and R. Bilik, "Case Study of Ransomware Malware Hiding Using Obfuscation Methods," in *2018 16th International Conference on Emerging eLearning Technologies and Applications (ICETA)*, Stary Smokovec, Slovakia, 2018.

[3] I. Tunji, N. Chomchoey, Phromchan and K. Chimmanee, "Ransomware Attack Analysis on Banking Systems," in *2023 7th International Conference on Information Technology (InCIT)*, Chiang Rai, Thailand, 2023.

[4] "Ransomware outbreak: Wannacry — Redsocks Security," [Online]. Available: https://www.redsocks.eu/news/ransomware-wannacry/.

[5] R. Sobers, "Ransomware Statistics, Data, Trends, and Facts [updated 2024]," varonis, 13 November 2024. [Online]. Available: https://www.varonis.com/blog/ransomware-statistics.

[6] M. Medhat, S. Gaber and N. Abdelbaki, "A New Static-Based Framework for Ransomware Detection," in *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech)*, Athens, Greece, 2018.

[7] M. Almousa, S. Basavaraju and M. Anwar, "API-Based Ransomware Detection Using Machine Learning-Based Threat Detection Models," in *2021 18th International Conference on Privacy, Security and Trust (PST)*, Auckland, New Zealand, 2021.

[8] S. R. Alvee, B. Ahn, T. Kim, Y. Su, Y. Youn and M. Ryu, "Ransomware Attack Modeling and Artificial Intelligence-Based Ransomware Detection for Digital Substations," in *2021 6th IEEE Workshop on the Electronic Grid (eGRID)*, New Orleans, LA, USA, 2021.

[9] F. Manavi and A. Hamzeh, "Static Detection of Ransomware Using LSTM," in *2021 26th International Computer Conference, Computer Society of Iran (CSICC)* , Tehran, Iran, 2021.

[10] I. Matin, "Static Portable Executable Analysis for Ransomware Classification Using Support Vector Machine," in *2020 IEEE Workshop on Electronic Grid (eGRID)*, Aachen, Germany, 2020.

[11] M.J.M.M.et al., "Gradient Boosting for Improved Static Ransomware Detection: Optimizing Real-time Inference," in *2022 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, Athens, Greece, 2022, 2022.

[12] S. Gulmez, A. Kakisim and I. Sogukpinar, "Analysis of the Dynamic Features on Ransomware Detection Using Deep Learning-based Methods," in *2023 11th International Symposium on Digital Forensics and Security (ISDFS)*, Chattanooga, TN, USA, 2023.

[13] M.J.M.M, U.S, M.B.P and S. Sandhya, "Detection of ransomware in static analysis by using Gradient Tree Boosting Algorithm," in *2020 International Conference on System, Computation, Automation and Networking (ICSCAN)*, Pondicherry, India, 2020.

[14] M. Medhat, M. Essa, H. Faisal and S. Sayed, "YARAMON: A Memory-based Detection Framework for Ransomware Families," in *2020 15th International Conference for Internet Technology and Secured Transactions (ICITST)*, London, United Kingdom, 2020.

[15] D. Netto, K. Shony and E. Lalson, "An Integrated Approach for Detecting Ransomware Using Static and Dynamic Analysis," in *2018 International CET Conference on Control, Communication, and Computing (IC4)*, Thiruvananthapuram, India, 2018.

[16] SentinelOne, "What Is RYUK Ransomware? A Detailed Breakdown," 16 July 2021. [Online]. Available: https://www.sentinelone.com/cybersecurity-101/threat-intelligence/ryuk-ransomware/.

[17] I.M.M.Matin, "Ransomware Extraction Using Static Portable Executable (PE) Feature-Based Approach," in *2023 6th International Conference of Computer and Informatics Engineering (IC2IE)*, Lombok, Indonesia, 2023.

[18] M. Kanwal, S. Thakur and R. Lashkari, "An app based on static analysis for android ransomware," in *2017 8th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, Delhi, India, 2017.

[19] F. Manavi and A. Hamzeh, "A New Method for Ransomware Detection Based on," in *17th International ISC Conference on Information Security and Cryptology (ISCISC'2020)*, Iran University of Science and Technology, 2020.